NEBIUS ACADEMY

LLM Architectures

Week 2

Gradient optimization

The objectives for today

1. How we train ML models
2. Evaluation
3. Regularization
4. Pytorch

NEBIUS ACADEMY

Gradient optimization

Two parts of the answer

We need to somehow get w:

$$f(x, w) = xw^T$$

Part 1: Loss function. We're searching for an optimal w, and the loss function tells us what we want to optimize.

$$\mathcal{L}(X_{train}, y_{train}, w_{better}) > \mathcal{L}(X_{train}, y_{train}, w_{worse})$$

Part 2: Optimization method. Tells us how do we find the optimal w.

A quick reminder about loss functions

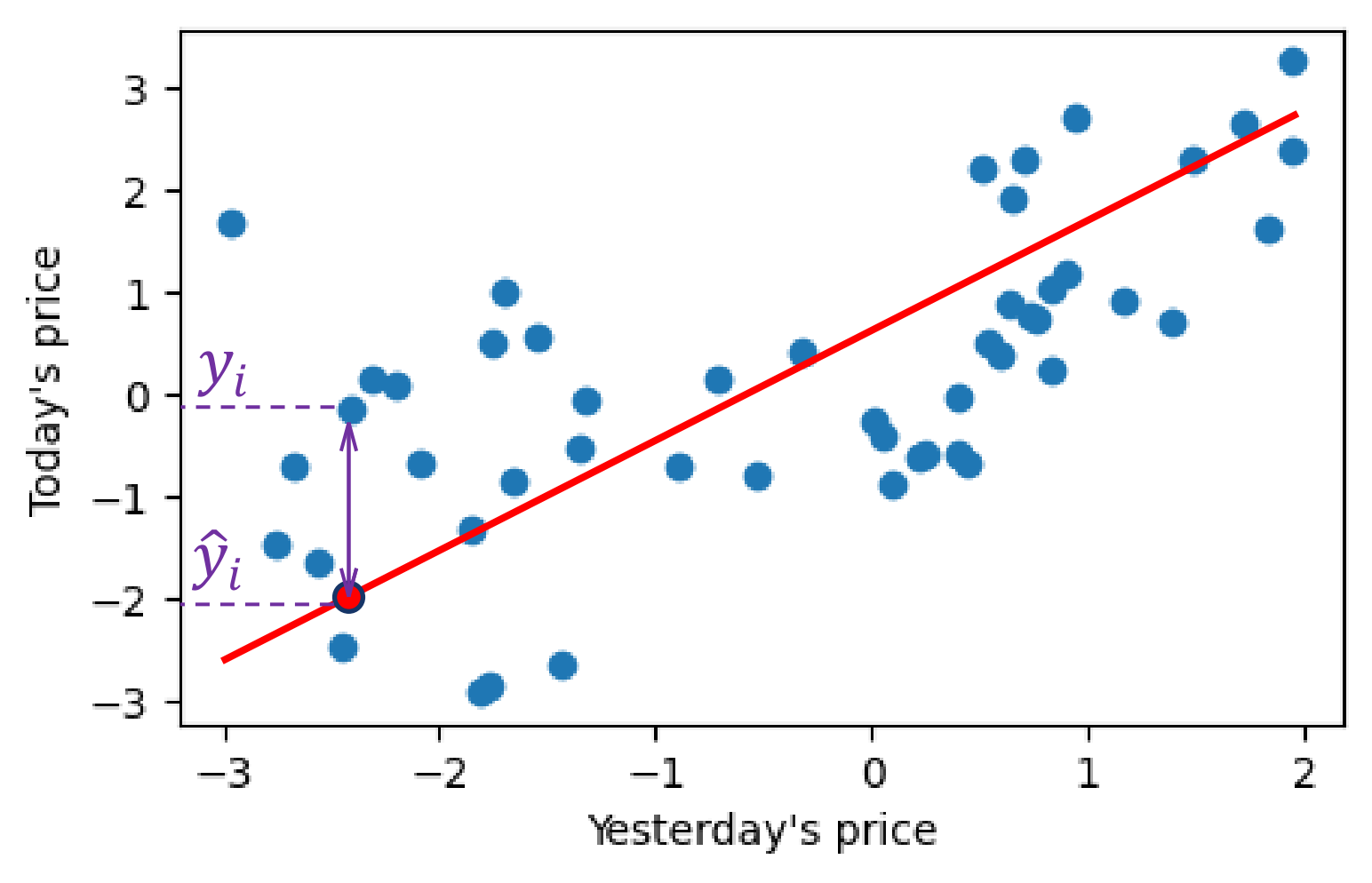
Regression loss

$$\mathcal{L}(y, \hat{y}) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}(y_i, \hat{y}_i)$$

y_i – true value
$\hat{y}_i$ – predicted value

MSE:

$$\mathcal{L}(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$$



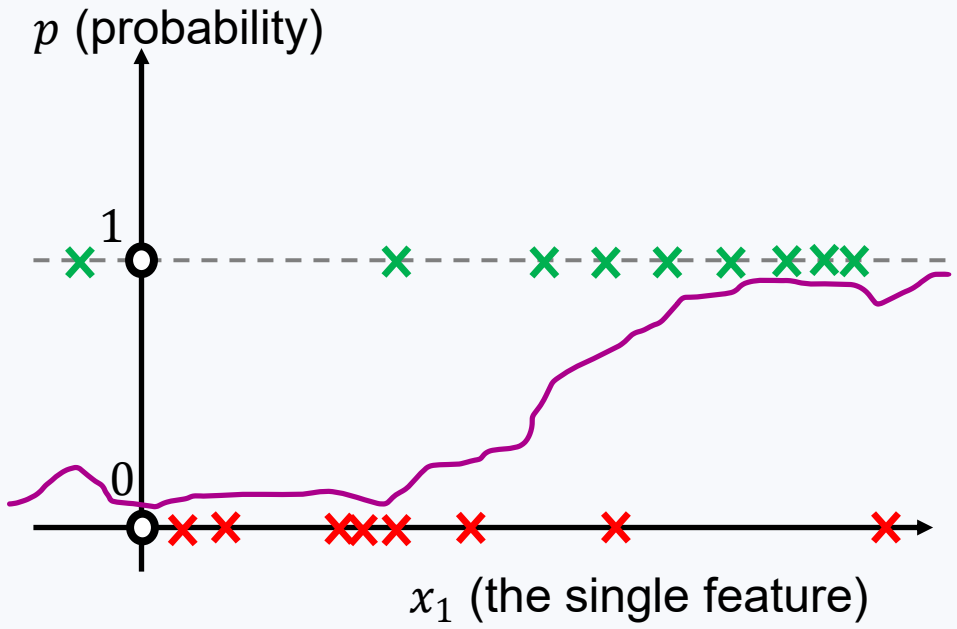
A quick reminder about loss functions

Classification:
cross-entropy

$$\mathcal{L}(y_i, \hat{p}_i) = -y_i \log \hat{p}_i - (1 - y_i) \log(1 - \hat{p}_i)$$

$$l(y_i, \hat{p}_i) = \begin{cases} -\log(1 - \hat{p}_i), & if\ y_i = 0 \\ -\log \hat{p}_i, & if\ y_i = 1 \end{cases}$$

y_i – true class labels
$\hat{p}_i$ – predicted probabilities



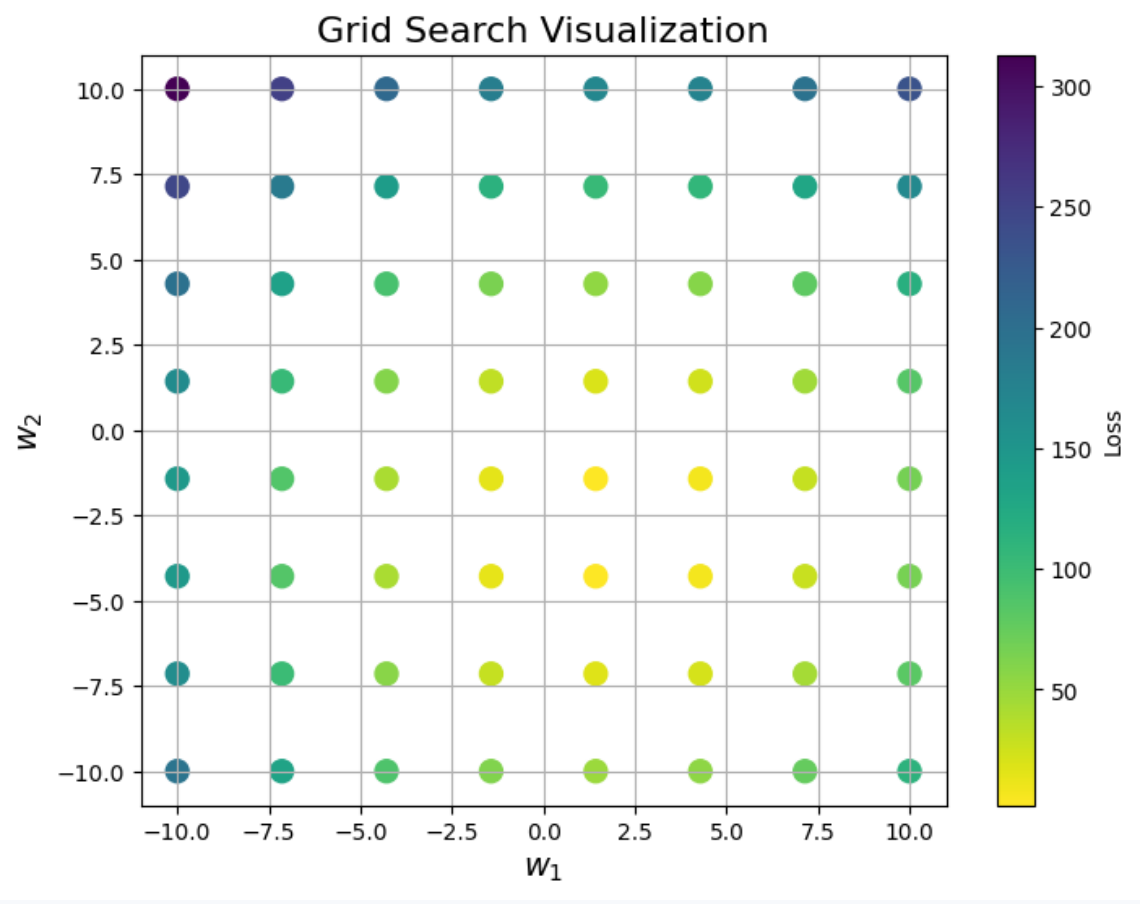
The next step – how to find the optimal w

We need to find the parameters w, which minimize the loss.

The next step – how to find the optimal w

We need to find the parameters w, which minimize the loss.

Why not grid search?



The next step – how to find the optimal w

We need to find the parameters w, which minimize the loss.

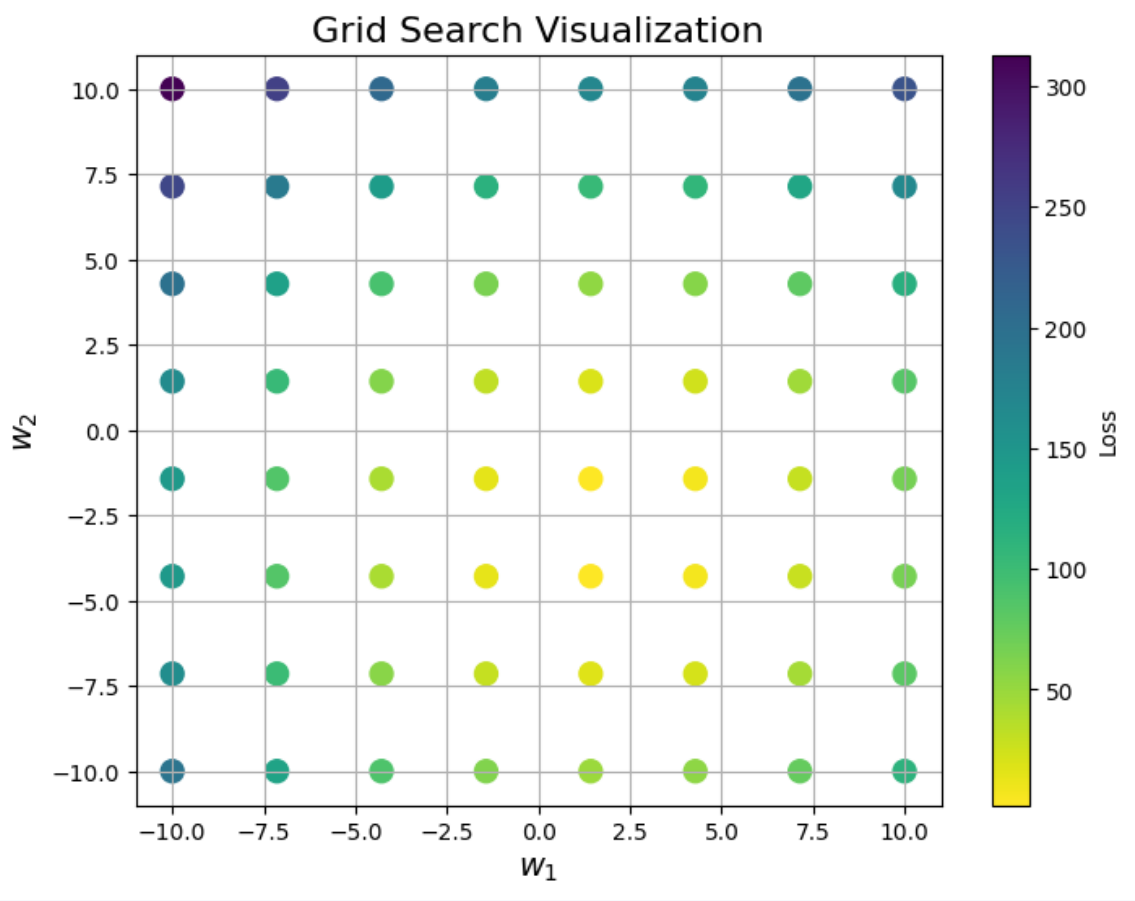
Why not grid search?

Imagine:

- 20 parameters
- -2..2 with 0.1 step (41 values)
- Training set size: 200
- Test set size: 100

We need to calculate $\mathcal{L}$ many times:

$41^{20} \cdot (200 + 100)$



NEBIUS ACADEMY

Gradients

What is a gradient

Gradient of $f(w_1, \dots, w_D)$ at a point $\mathbf{w}_0 = (w_{01}, \dots, w_{0D})$ is the vector

$$[\nabla_w f](w_0) = \left(\frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_D}\right)\Bigg|_{w=w_0}$$

Example:

$$f(w_1, \dots, w_D) = \sin(w_1) + \exp(w_1 w_2)$$

$$\frac{\partial f}{\partial w_1} = \cos(w_1) + w_2 \exp(w_1 w_2)$$

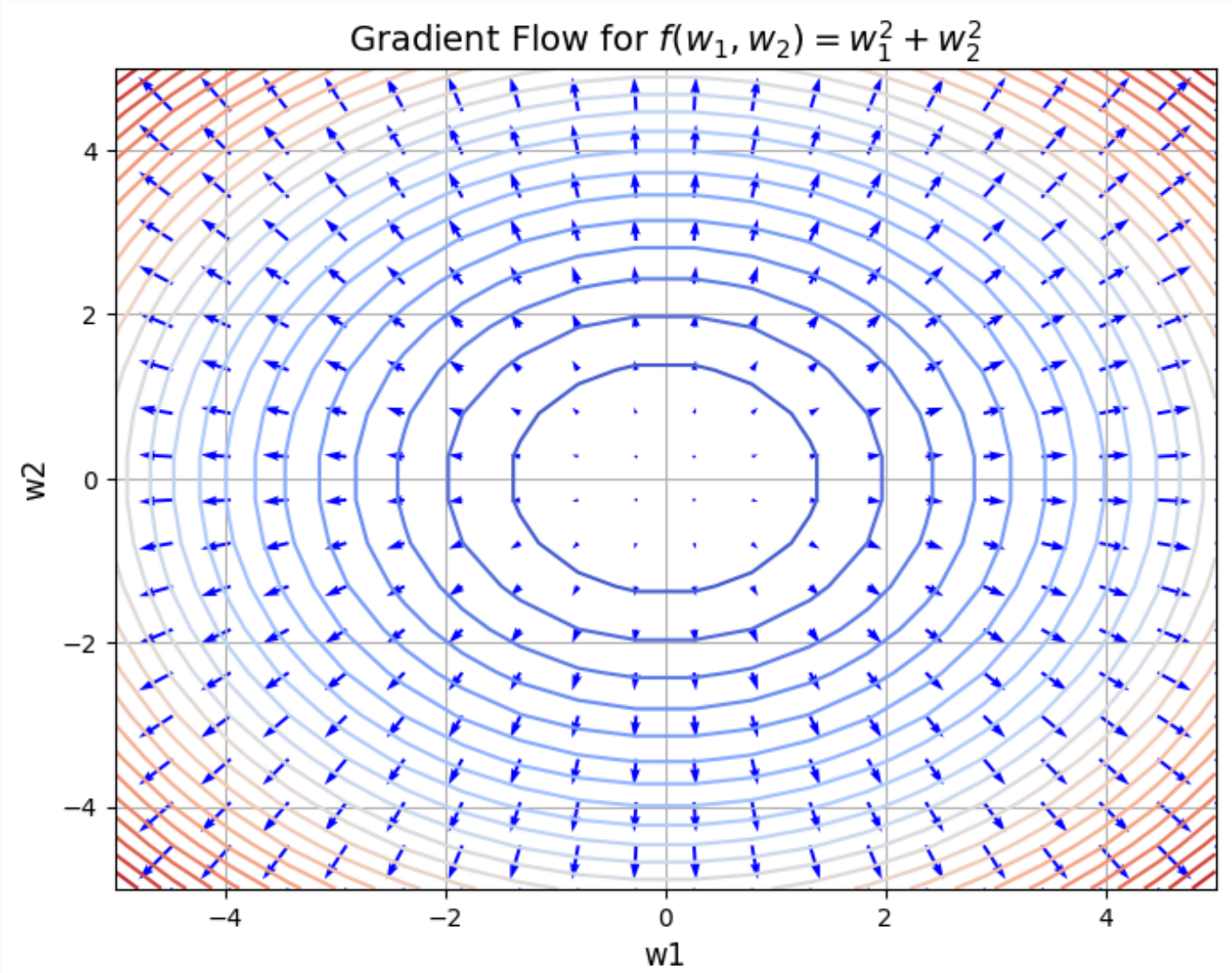
$$\frac{\partial f}{\partial w_2} = w_1 \exp(w_1 w_2)$$

$$[\nabla_w f](0, 1) = (2,0)$$

What is a gradient

Gradient of $f(w_1, \dots, w_D)$ at a point $\mathbf{w}_0 = (w_{01}, \dots, w_{0D})$ is the vector

$$[\nabla_w f](w_0) = \left(\frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_D}\right)\Bigg|_{w=w_0}$$

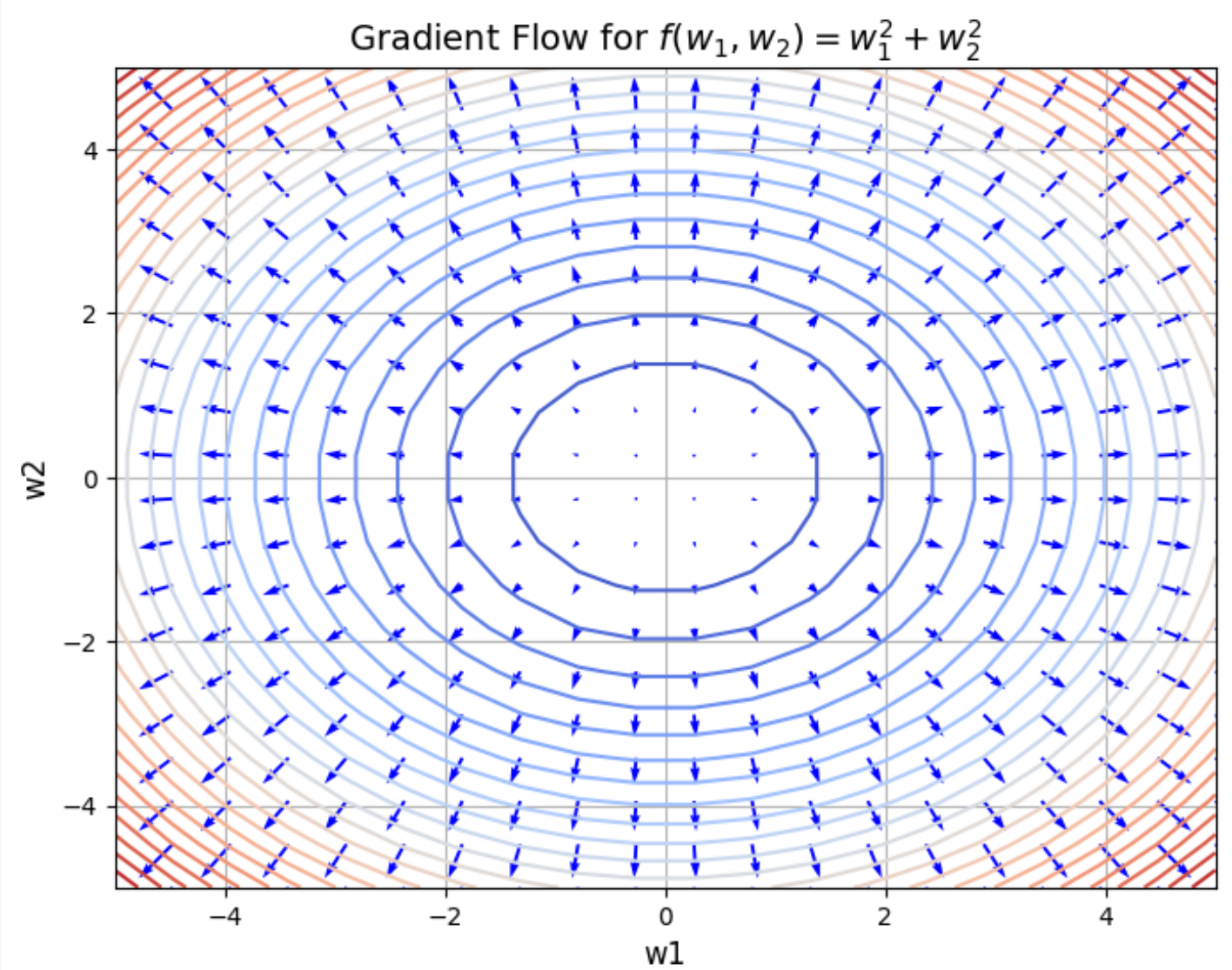


What is a gradient

Gradient of $f(w_1, \dots, w_D)$ at a point $\mathbf{w}_0 = (w_{01}, \dots, w_{0D})$ is the vector

$$[\nabla_w f](w_0) = \left(\frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_D}\right)\Bigg|_{w=w_0}$$

It is the **direction in which the function increases the fastest**



What is a gradient

Gradient of $f(w_1, \dots, w_D)$ at a point $\mathbf{w}_0 = (w_{01}, \dots, w_{0D})$ is the vector

$$[\nabla_w f](w_0) = \left(\frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_D}\right)\Bigg|_{w=w_0}$$

It is the **direction in which the function increases the fastest**

$$f(w_0 + h) = f(w_0) + \langle \nabla_w f(w_0), h \rangle + \dots$$

If $\|h\| = d$:

$$\langle \nabla_w f(w_0), h \rangle = \|\nabla_w f(w_0)\| \cdot d \cdot \cos \angle(\nabla_w f(w_0), h)$$

The maximum is when h and $\nabla_w f(w_0)$ have the same direction.

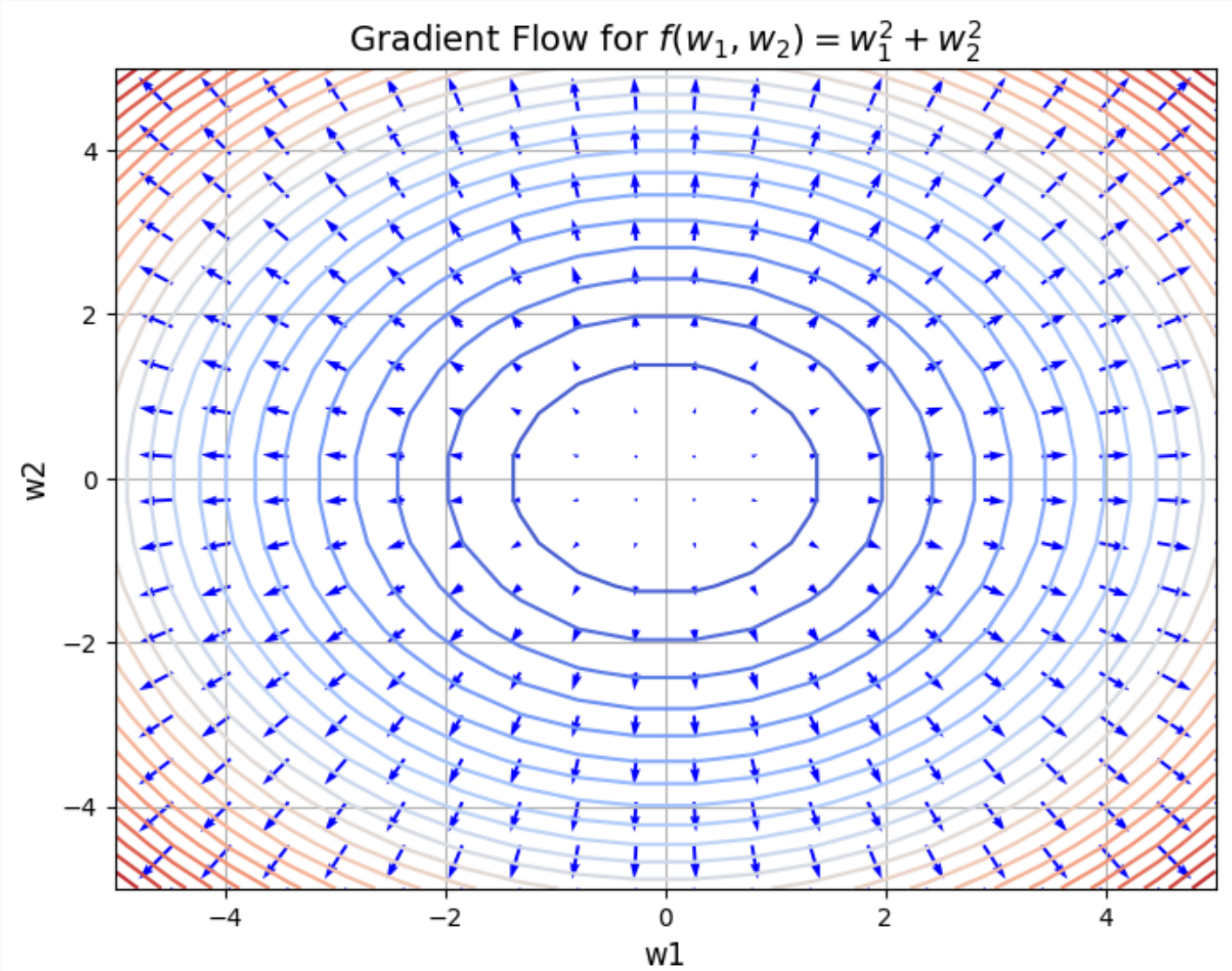
An obvious application

Loss minimum is where the gradient is zero

Solve

$$\frac{\partial f}{\partial w_1} = \cdots = \frac{\partial f}{\partial w_D} = 0,$$

get the answer.



How it works

Loss minimum is where the gradient is zero

Solve

$$\frac{\partial f}{\partial w_1} = \cdots = \frac{\partial f}{\partial w_D} = 0,$$

get the answer.

How it works

Linear regression

$$\mathcal{L}(X, w, y) = \sum_{i=1}^{N} \left(y_i - x_i w^T\right)^2$$

$$= \sum_{i=1}^{N} \left(y_i - \sum_{j=1}^{D} x_{ij} w_j \right)^2$$

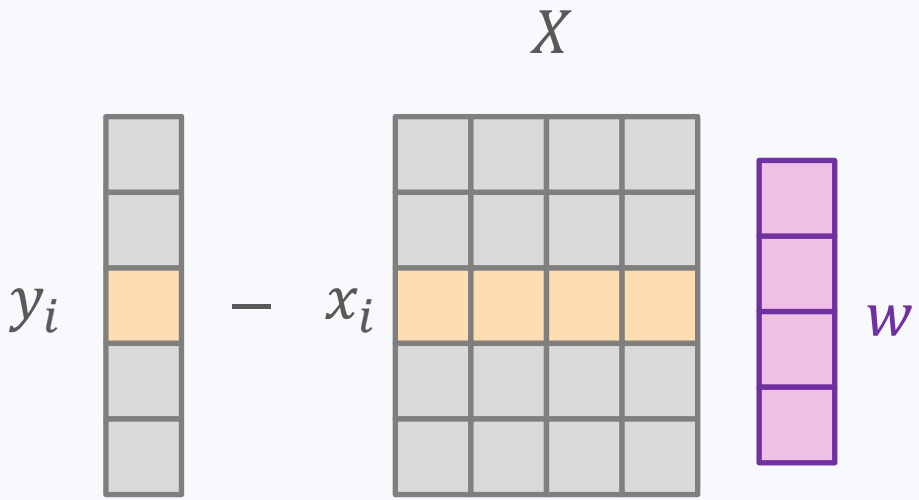
Loss minimum is where the gradient is zero

Solve

$$\frac{\partial f}{\partial w_1} = \cdots = \frac{\partial f}{\partial w_D} = 0,$$

get the answer.

How it works



Linear regression

$$\mathcal{L}(X, w, y) = \sum_{i=1}^{N} \left(y_i - x_i w^T\right)^2$$

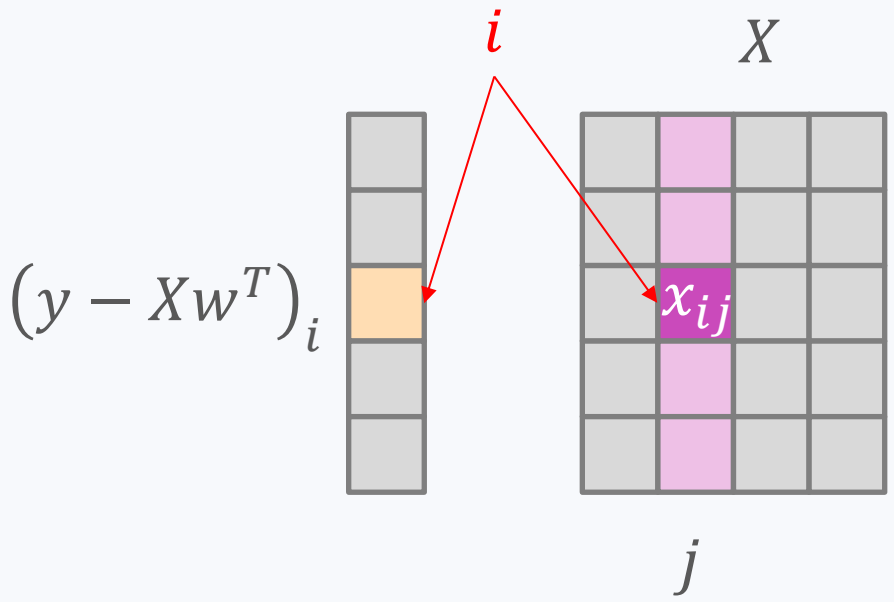
$$= \sum_{i=1}^{N} \left(y_i - \sum_{j=1}^{D} x_{ij} w_j \right)^2$$

$$\frac{\partial \mathcal{L}}{\partial w_j} = \sum_{i=1}^{N} 2\left(y_i - x_i w^T\right) \cdot \left(-x_{ij}\right) =$$

$$= -2 \sum_{i=1}^{N} \left(y - X w^T\right)_i \cdot x_{ij} =$$

How it works

Linear regression



$$\mathcal{L}(X, w, y) = \sum_{i=1}^{N} \left(y_i - x_i w^T\right)^2$$

$$= \sum_{i=1}^{N} \left(y_i - \sum_{j=1}^{D} x_{ij} w_j \right)^2$$

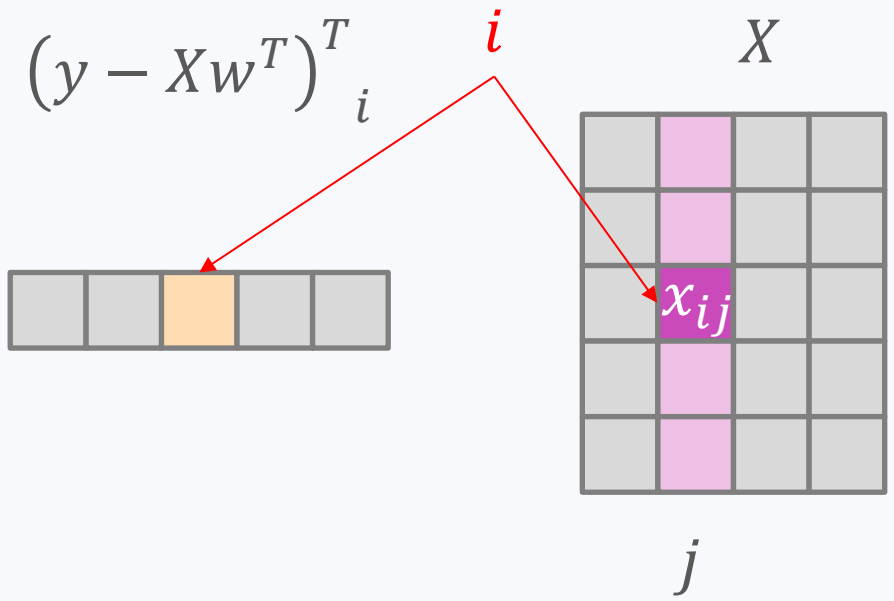
$$\frac{\partial \mathcal{L}}{\partial w_j} = \sum_{i=1}^{N} 2\left(y_i - x_i w^T\right) \cdot \left(-x_{ij}\right) =$$

$$= -2 \sum_{i=1}^{N} \left(y - Xw^T\right)_i \cdot x_{ij}$$

$$= \left[-2\left(y - Xw^T\right)^T X\right]_j$$

How it works

Linear regression



$$\mathcal{L}(X, w, y) = \sum_{i=1}^{N} \left(y_i - x_i w^T\right)^2$$

$$= \sum_{i=1}^{N} \left(y_i - \sum_{j=1}^{D} x_{ij} w_j \right)^2$$

$$\frac{\partial \mathcal{L}}{\partial w_j} = \sum_{i=1}^{N} 2\left(y_i - x_i w^T\right) \cdot \left(-x_{ij}\right) =$$

$$= -2 \sum_{i=1}^{N} \left(y - Xw^T\right)_i \cdot x_{ij}$$

$$= \left[-2\left(y - Xw^T\right)^T X\right]_j$$

How it works

Linear regression

$$\mathcal{L}(X, w, y) = \sum_{i=1}^{N} \left(y_i - x_i w^T\right)^2$$

$$= \sum_{i=1}^{N} \left(y_i - \sum_{j=1}^{D} x_{ij} w_j \right)^2$$

$$\frac{\partial \mathcal{L}}{\partial w_j} = \sum_{i=1}^{N} 2\left(y_i - x_i w^T\right) \cdot \left(-x_{ij}\right) =$$

$$= -2 \sum_{i=1}^{N} \left(y - X w^T\right)_i \cdot x_{ij}$$

$$= \left[-2\left(y - X w^T\right)^T X\right]_j$$

$$\nabla_w \mathcal{L} = -2\left(y - X w^T\right)^T X$$

Solution: $w^T = \left(X^T X\right)^{-1} X^T y$

Logistic regression

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \frac{1}{(1 + e^{-z})^2}(e^{-z}) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z} + 1 - 1}{1 + e^{-z}} =$$

$$= \sigma(z)(1 - \sigma(z))$$

Logistic regression

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \frac{1}{(1 + e^{-z})^2}(e^{-z}) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z} + 1 - 1}{1 + e^{-z}} =$$

$$= \sigma(z)(1 - \sigma(z))$$

$$\log \sigma(z)' = \frac{1}{\sigma(z)} \sigma(z)\big(1 - \sigma(z)\big) = 1 - \sigma(z)$$

$$\log\big(1 - \sigma(z)\big)' = \frac{1}{1 - \sigma} \cdot -\sigma(1 - \sigma) = -\sigma(z)$$

Logistic regression

$$\log \sigma(z)' = 1 - \sigma(z)$$
$$\log\big(1 - \sigma(z)\big)' = -\sigma(z)$$

$$\mathcal{L}(X, w, y) = -\sum_{i=1}^{N} (1 - y_i) \log\left(1 - \sigma\left(\sum_{j=1}^{D} x_{ij} w_j\right)\right) + y_i \log \sigma\left(\sum_{j=1}^{D} x_{ij} w_j\right) =$$

Logistic regression

$$\log \sigma(z)' = 1 - \sigma(z)$$
$$\log\big(1 - \sigma(z)\big)' = -\sigma(z)$$

$$\mathcal{L}(X, w, y) = -\sum_{i=1}^{N} (1 - y_i) \log\left(1 - \sigma\left(\sum_{j=1}^{D} x_{ij} w_j\right)\right) + y_i \log \sigma\left(\sum_{j=1}^{D} x_{ij} w_j\right) =$$

$$\frac{\partial \mathcal{L}}{\partial w_j} = -\sum_{i=1}^{N} (1 - y_i) \cdot -\sigma\big(x_i w^T\big) x_{ij} + y_i \left(1 - \sigma\big(x_j w^T\big)\right) x\_ij =$$

Logistic regression

$$\log \sigma(z)' = 1 - \sigma(z)$$
$$\log\big(1 - \sigma(z)\big)' = -\sigma(z)$$

$$\mathcal{L}(X, w, y) = -\sum_{i=1}^{N}(1 - y_i)\log\left(1 - \sigma\left(\sum_{j=1}^{D} x_{ij} w_j\right)\right) + y_i \log\sigma\left(\sum_{j=1}^{D} x_{ij} w_j\right) =$$

$$\frac{\partial \mathcal{L}}{\partial w_j} = -\sum_{i=1}^{N}(1 - y_i) \cdot -\sigma\big(x_i w^T\big) x_{ij} + y_i \Big(1 - \sigma\big(x_j w^T\big)\Big) x\_ij =$$

$$= -\sum_{i=1}^{N}\big[-\sigma\big(x_i w^T\big) + y_i(\sigma - \sigma + 1)\big] x_{ij} = -\sum_{i=1}^{N}\big[y_i - \sigma\big(x_i w^T\big)\big] x_{ij} =$$

$$= -\Big(y - \sigma\big(X w^T\big)\Big)^T X$$

Compare:

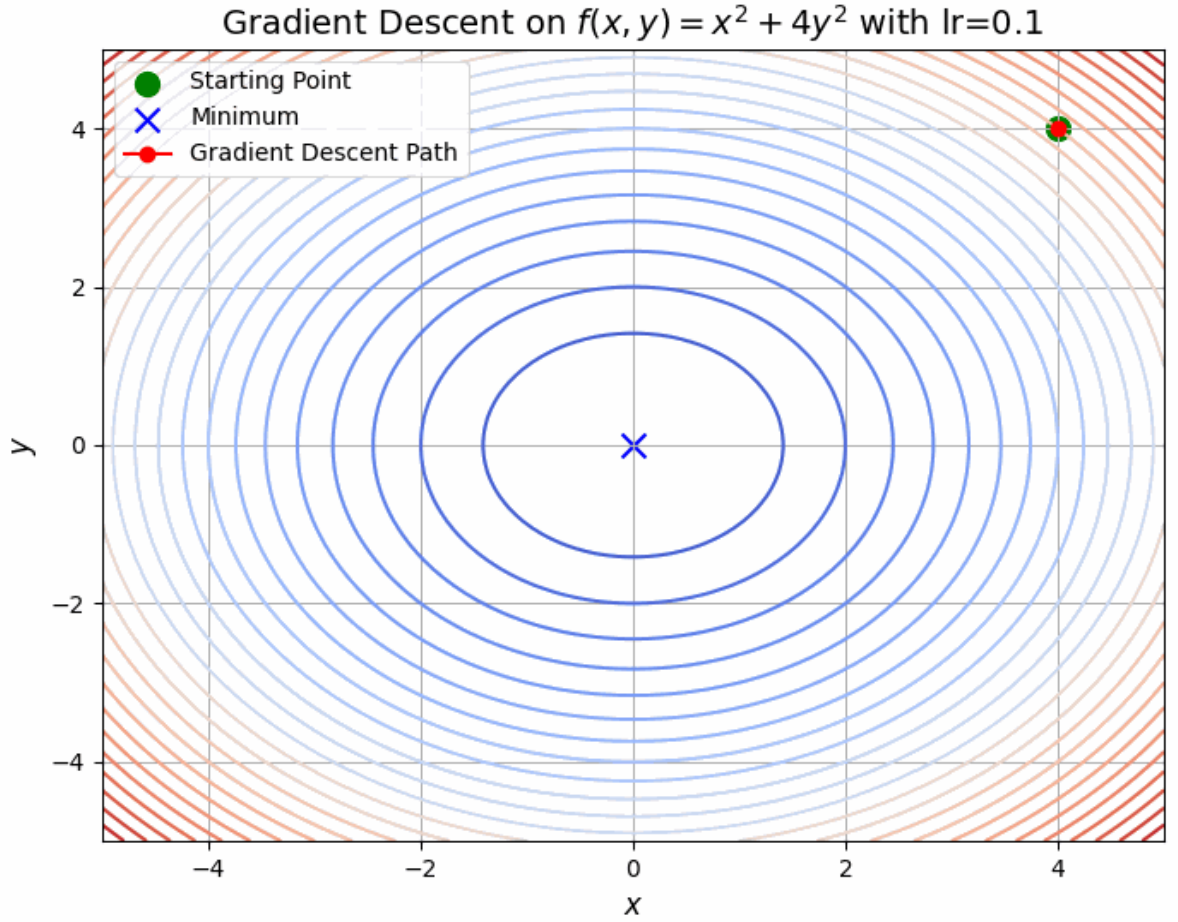
$$-2\big(y - X w^T\big)^T X$$

We can't solve it ☹

Gradient descent

Gradient is the direction of the steepest increase ⇒ **Anti-gradient $-\nabla f$ is the direction of the steepest decrease**

Just go along the anti-gradient to reach the minimum!

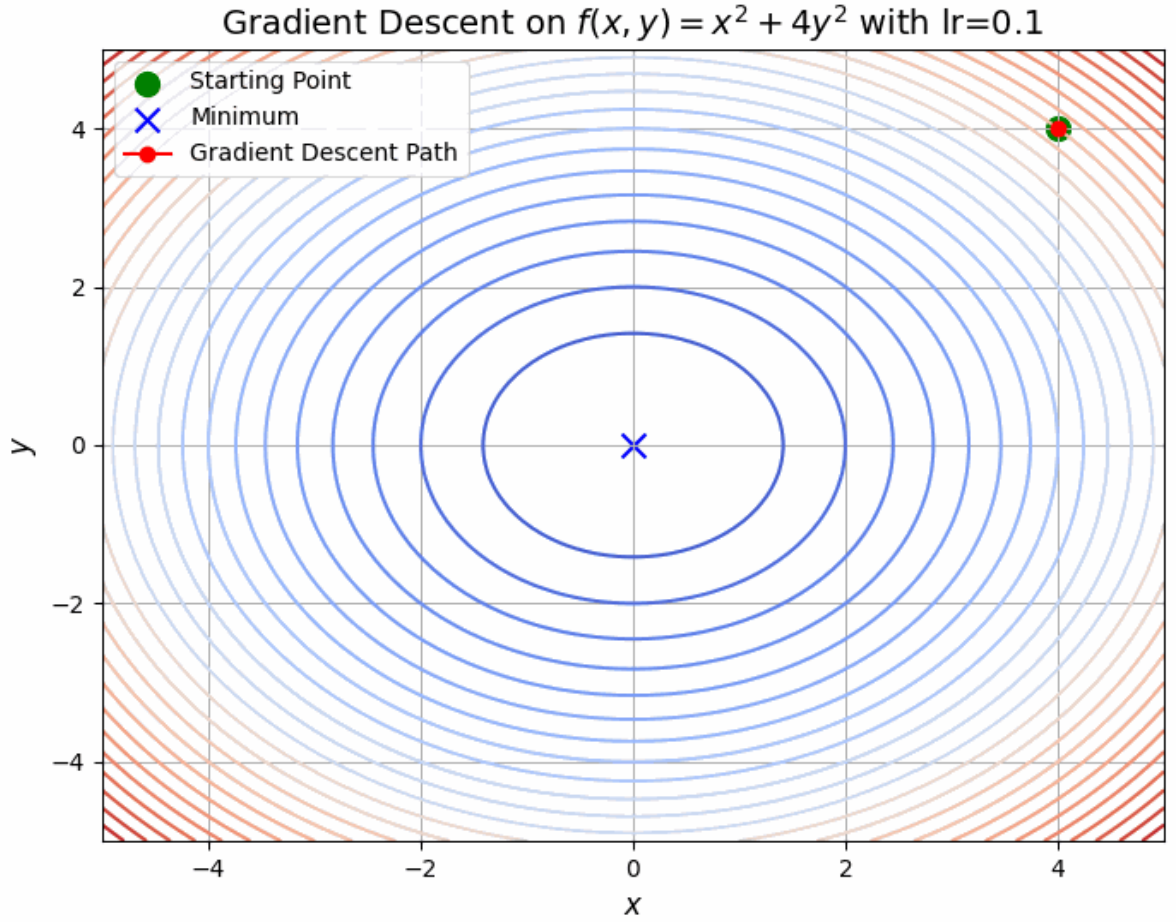


Gradient descent

Math formulation:

Given: f, ∇f, w_0, **max_steps**,
α – learning rate

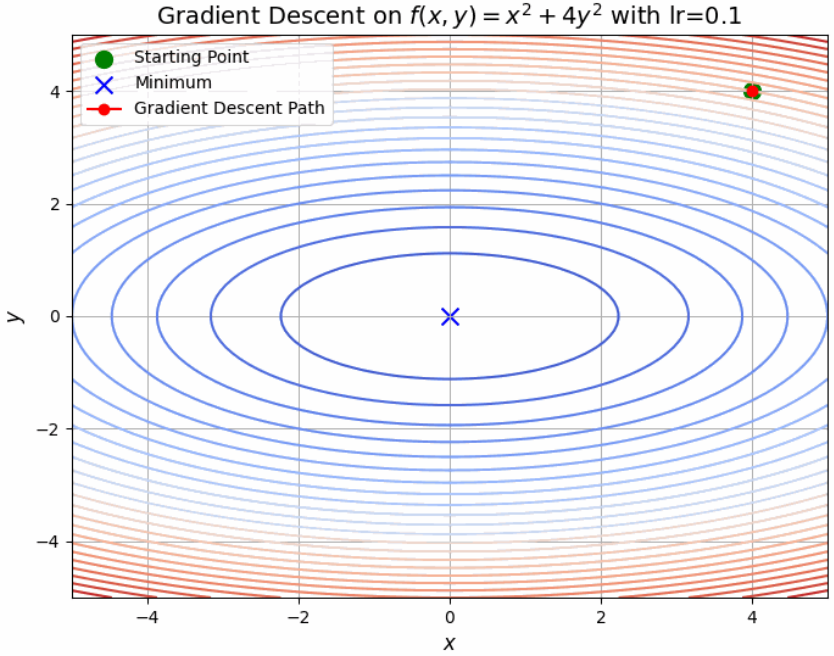
1. Initialize w with w_0
2. Perform several anti-gradient steps:
$$w_{m+1} = w_m - \alpha \cdot \nabla_w f(w_m)$$
3. Finish when reached **max_steps** or converged

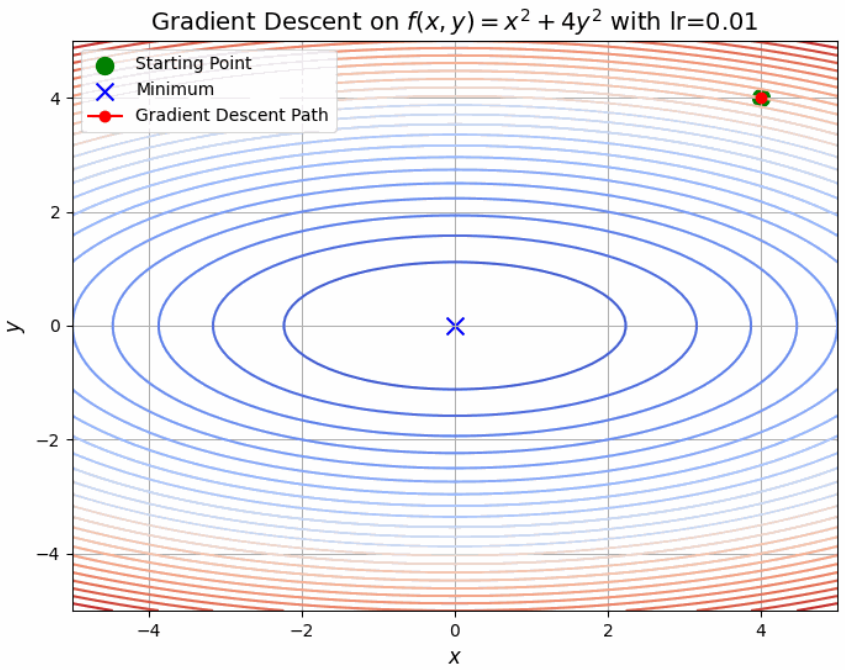


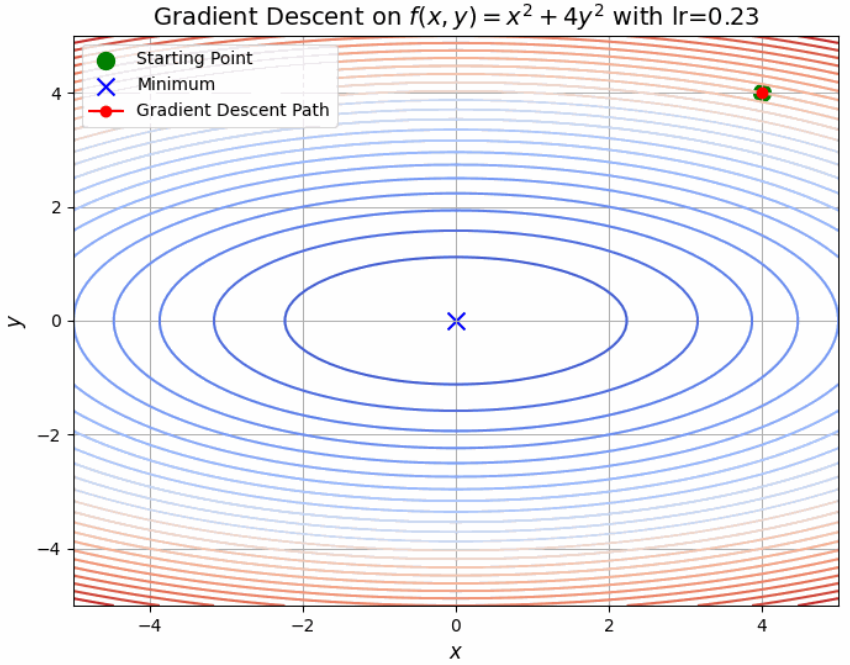
Learning rate α is important

If it's too little, it won't converge;

If it's too big, it will diverge.







NEBIUS ACADEMY

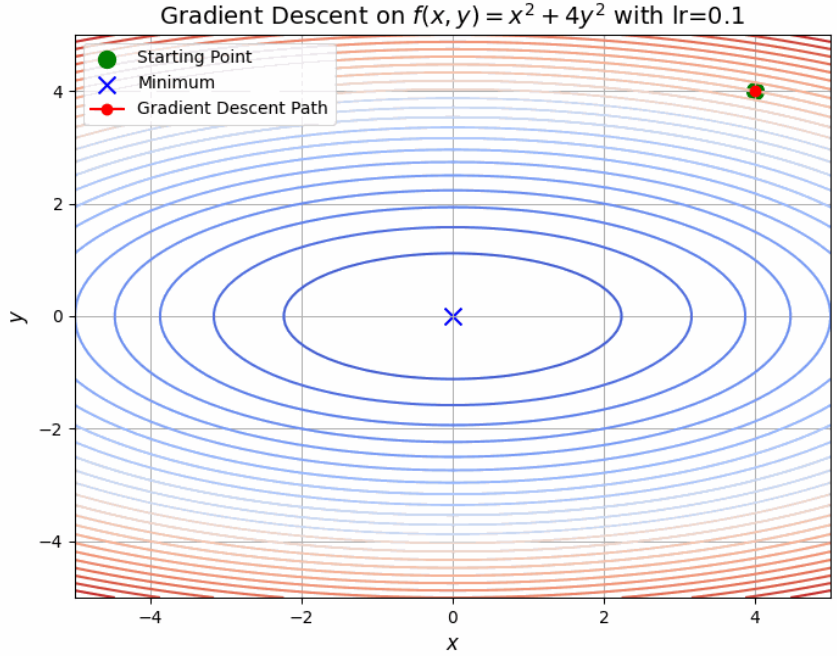
Learning rate is important

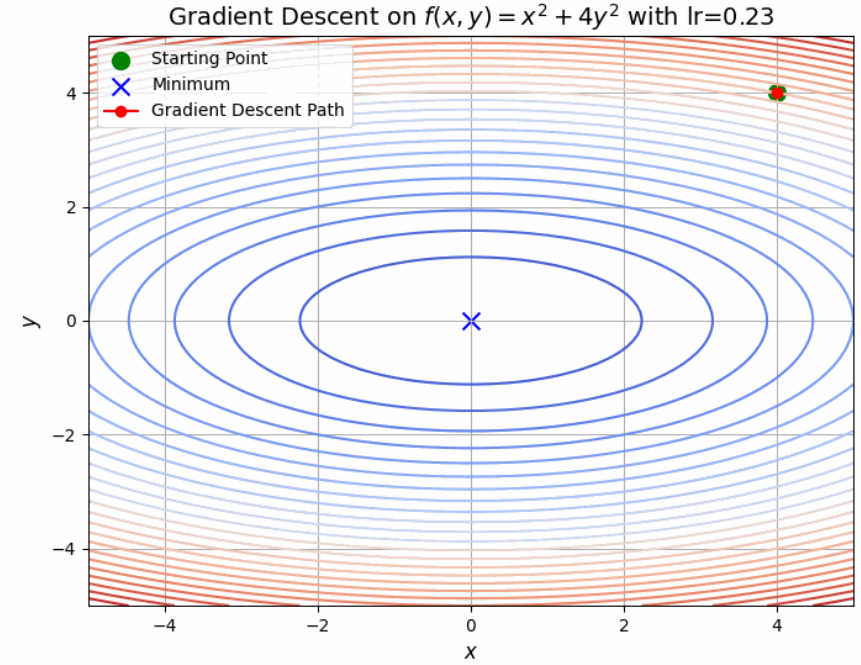
If it's too little, it won't converge;

If it's too big, it will diverge.

Learning rate is a **hyperparameter** of optimization tuned on a logarithmic scale. The choice would be between: 1e-5, 1e-4, 1e-3, 1e-2

In practice, learning rate decay is often used.





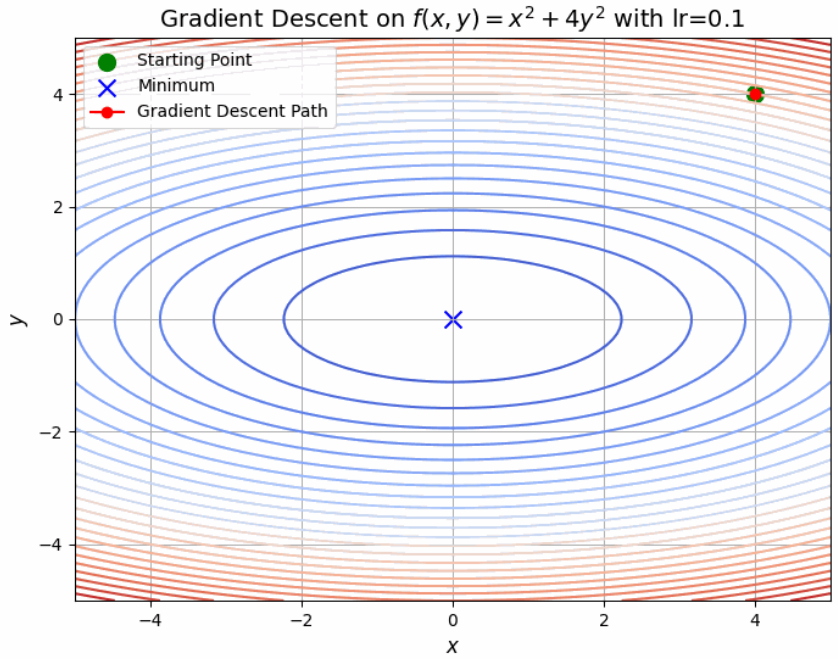
About elongated loss basin

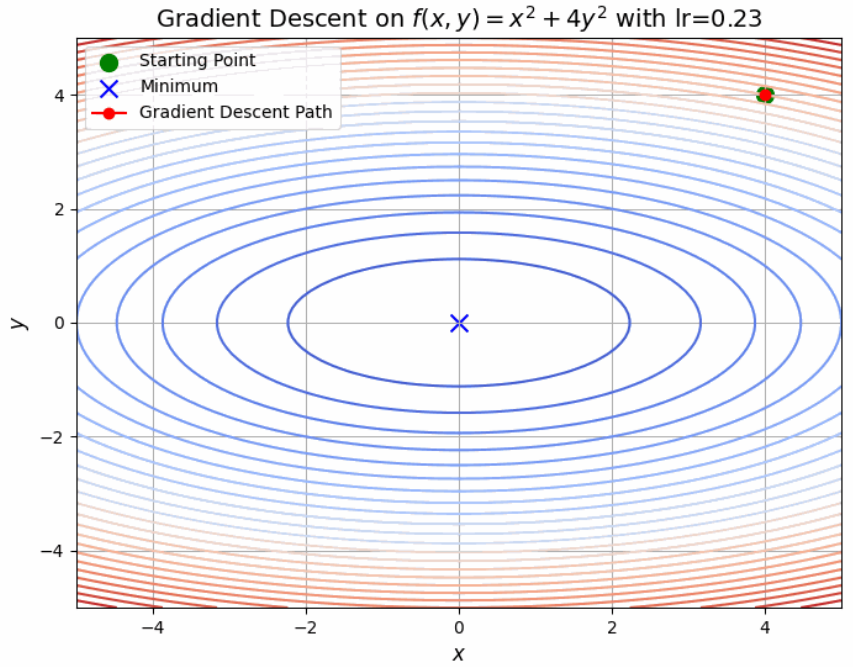
$$\mathcal{L}(X, w, y) = \sum_{i=1}^{N} \left(y_i - x_i w^T\right)^2 =$$

$$= \left\|y - Xw^T\right\|^2 = \left(y - Xw^T\right)^T \left(y - Xw^T\right) =$$

$$= \left(y^T - wX^T\right)\left(y - Xw^T\right) =$$

$$= wX^T Xw^T + linear\ terms$$





About elongated loss basin

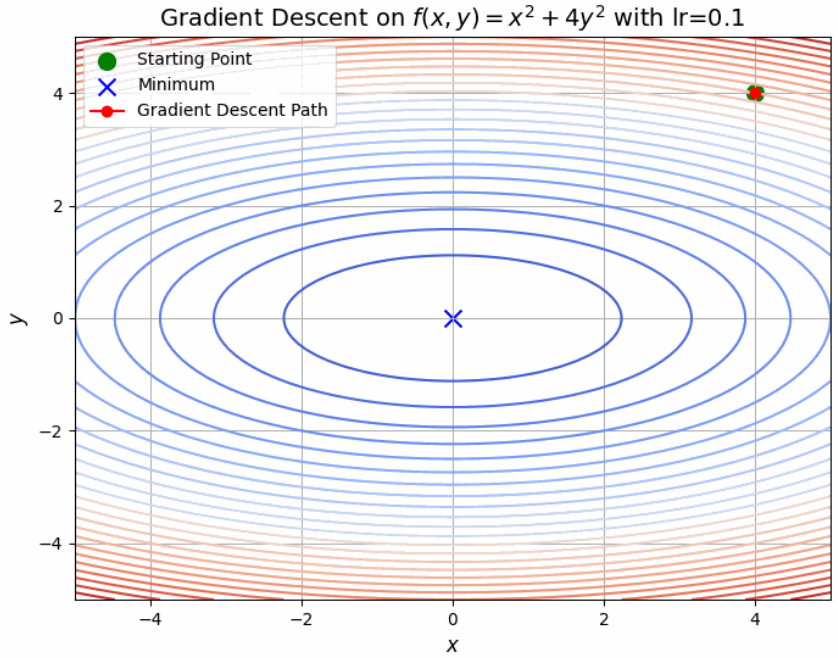
$$\mathcal{L}(X, w, y) = \sum_{i=1}^{N} \left(y_i - x_i w^T\right)^2 =$$

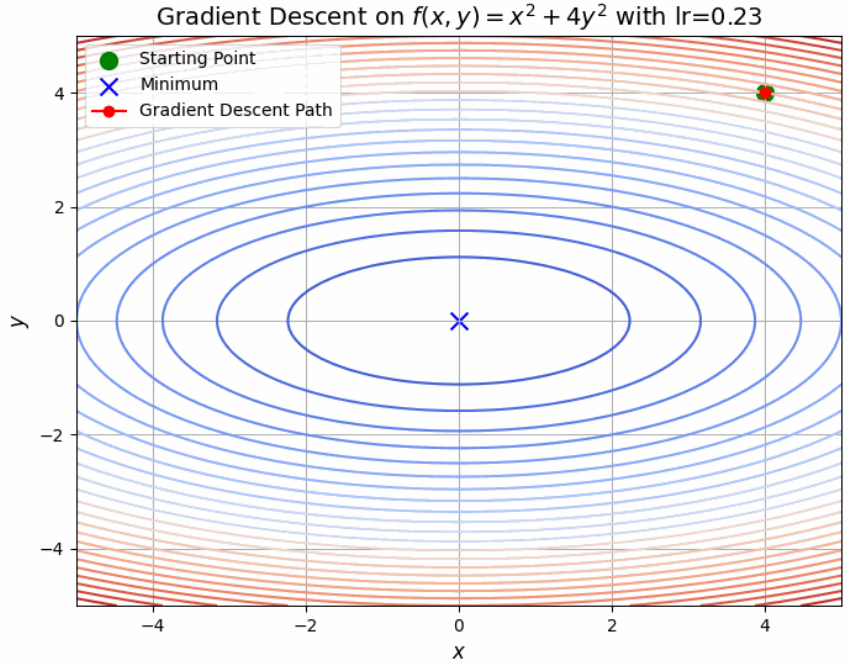
$$= wX^T Xw^T + linear\ terms$$

If every feature has $mean = 0$, then

$$\left(X^T X\right)_{ij} \sim Cov\left(feature_i, feature_j\right)$$

$$\left(X^T X\right)_{ii} \sim \mathbb{V}\left(feature_i\right)$$





About elongated loss basin

$$\mathcal{L}(X, w, y) = \sum_{i=1}^{N} \left(y_i - x_i w^T\right)^2 =$$

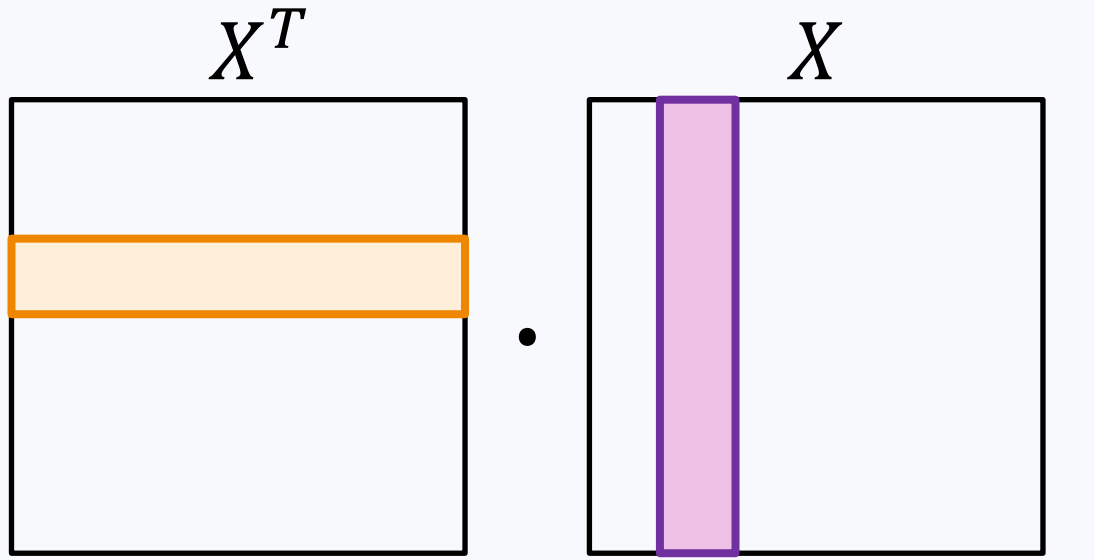
$$= wX^T Xw^T + linear\ terms$$

If every feature has $mean = 0$, then

$$\left(X^T X\right)_{ij} \sim Cov\left(feature_i, feature_j\right)$$

$$\left(X^T X\right)_{ii} \sim \mathbb{V}\left(feature_i\right)$$

$$\left(X^T X\right)_{ij} =$$



i-th row of X^T
i-th column of X

j-th column of X

About elongated loss basin

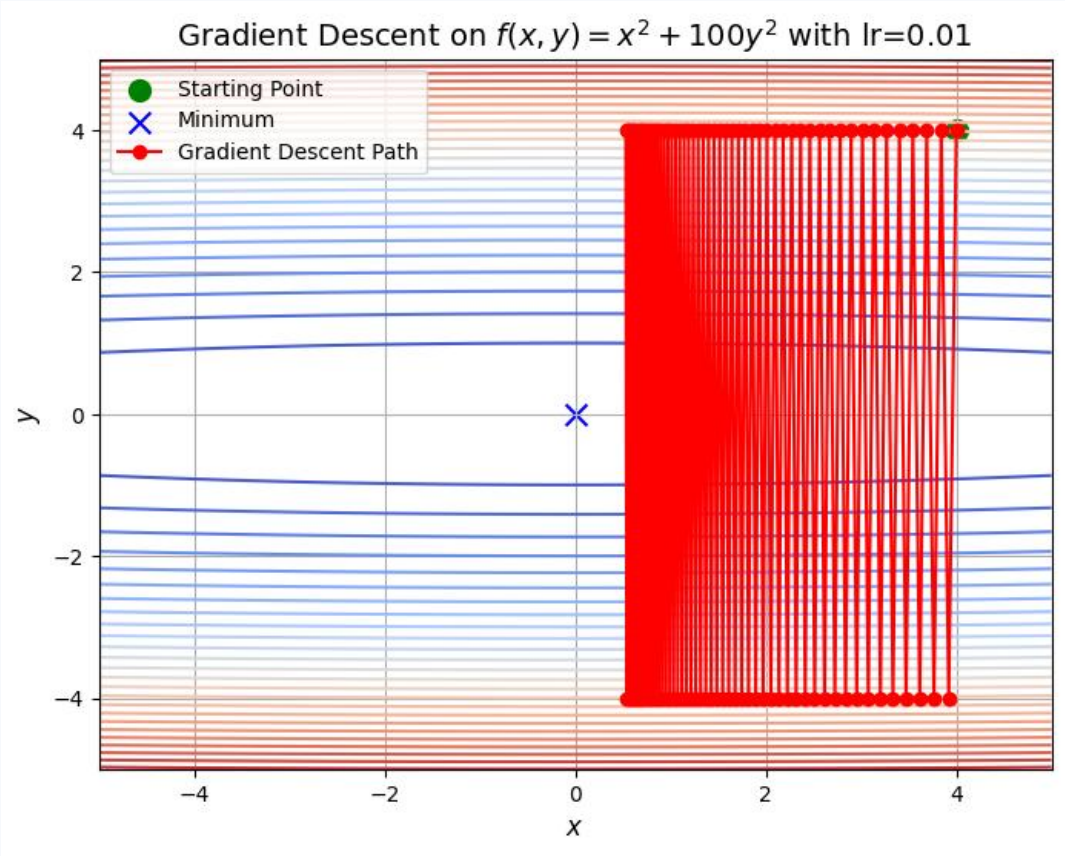
$$\mathcal{L}(X, w, y) = \sum_{i=1}^{N} \left(y_i - x_i w^T\right)^2 =$$

$$= wX^T Xw^T + linear\ terms$$

If every feature has $mean = 0$, then

$$\left(X^T X\right)_{ij} \sim Cov\left(feature_i, feature_j\right)$$

$$\left(X^T X\right)_{ii} \sim \mathbb{V}(feature_i)$$



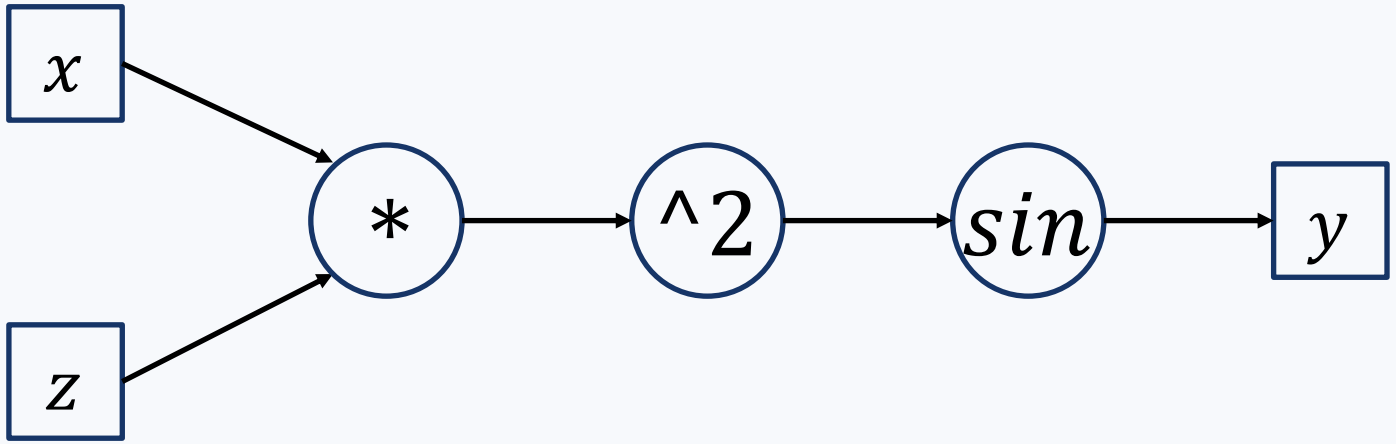
NEBIUS ACADEMY

Autodiff

Automating differentiation

Every expression is a computational graph

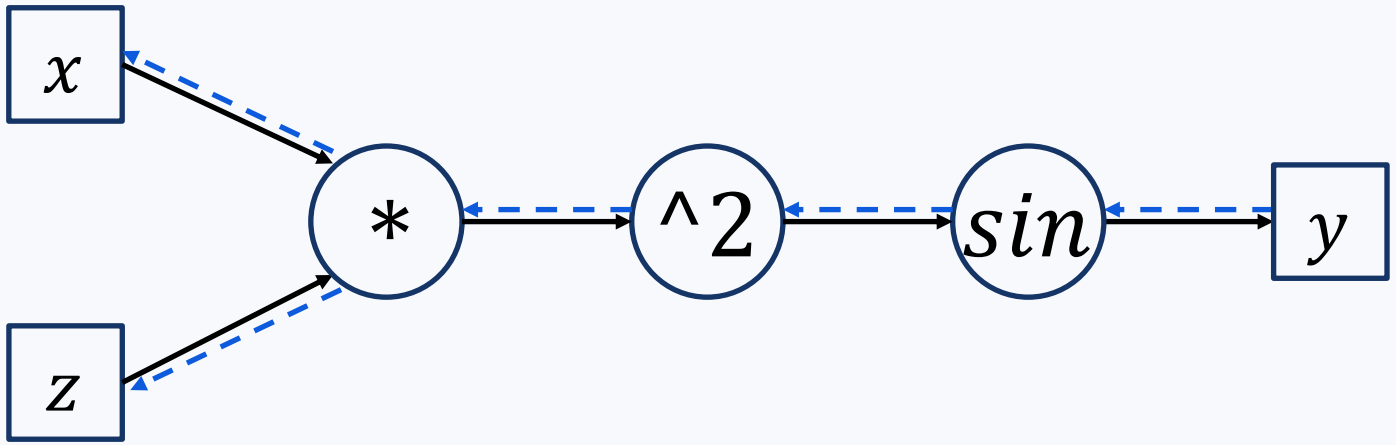
Example. $f(x, z) = \sin(x \cdot z)^2$



Automating differentiation

Every expression is a computational graph

Example. $f(x, z) = \sin(x \cdot z)^2$

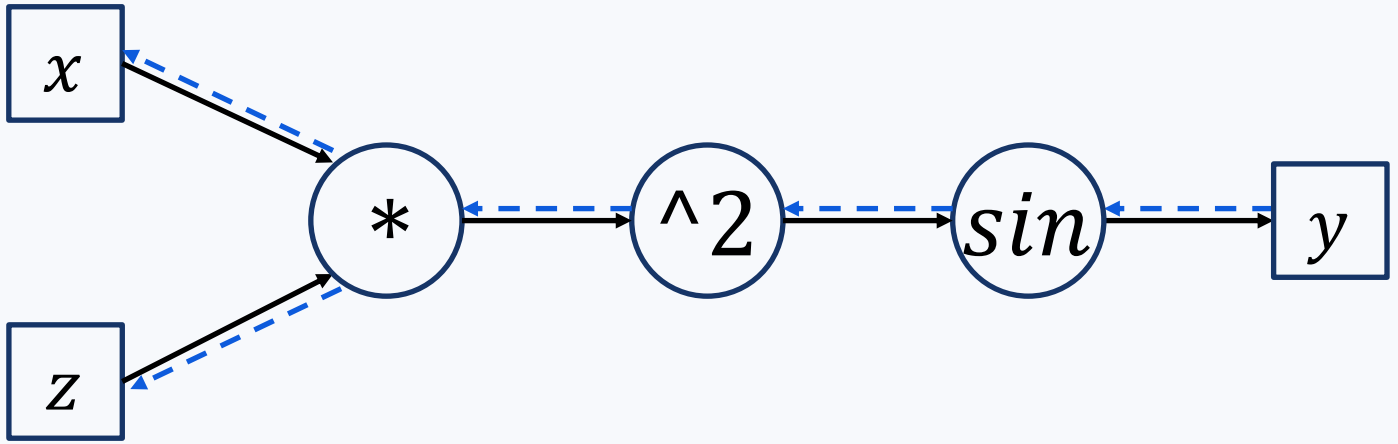


Derivatives flow backward

Automating differentiation

Every expression is a computational graph

Example. $f(x, z) = \sin(x \cdot z)^2$



Derivatives flow backward

$$\frac{\partial}{\partial x} \sin(x \cdot z)^2 \Big|_{\substack{x=1 \\ z=2}}$$

$$= \cos\left((x \cdot z)^2 \Big|_{\substack{x=1 \\ z=2}} \right)$$

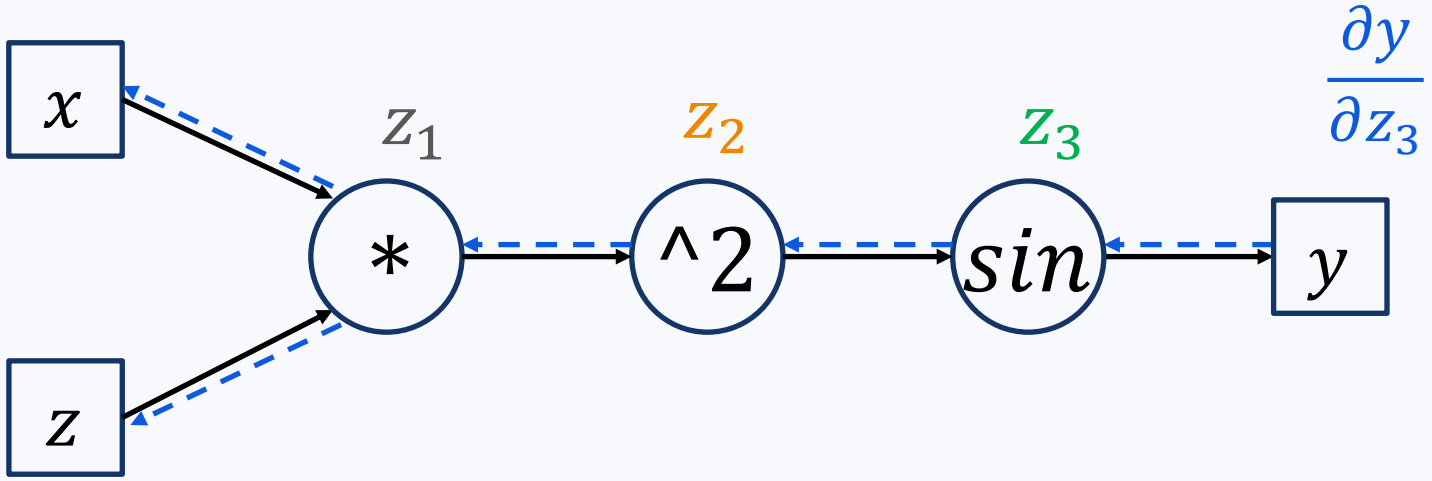
$$\cdot 2 \left((x \cdot z) \Big|_{\substack{x=1 \\ z=2}} \right)$$

$$\cdot z$$

Automating differentiation

Every expression is a computational graph

Example. $f(x, z) = \sin(x \cdot z)^2$



Derivatives flow backward

$$\frac{\partial}{\partial x} \sin(x \cdot z)^2 \Big|_{\substack{x=1 \\ z=2}}$$

$$= \cos\left((x \cdot z)^2 \Big|_{\substack{x=1 \\ z=2}} \right) \cdot 2 \left((x \cdot z) \Big|_{\substack{x=1 \\ z=2}} \right) \cdot z$$

$$\frac{\partial y}{\partial z_2} = \frac{\partial y}{\partial z_3}(z_2) \cdot \frac{\partial z_3}{\partial z_2}$$

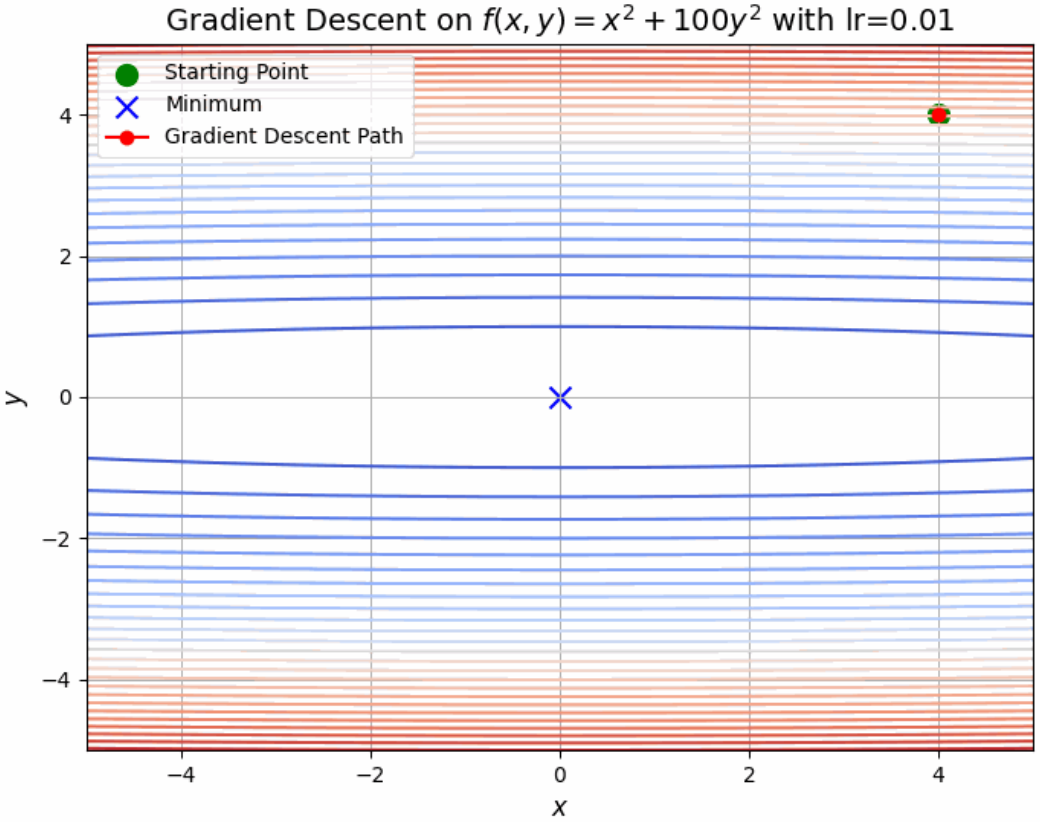
$$\frac{\partial y}{\partial z_1} = \frac{\partial y}{\partial z_2}(z_1) \cdot \frac{\partial z_2}{\partial z_1}$$

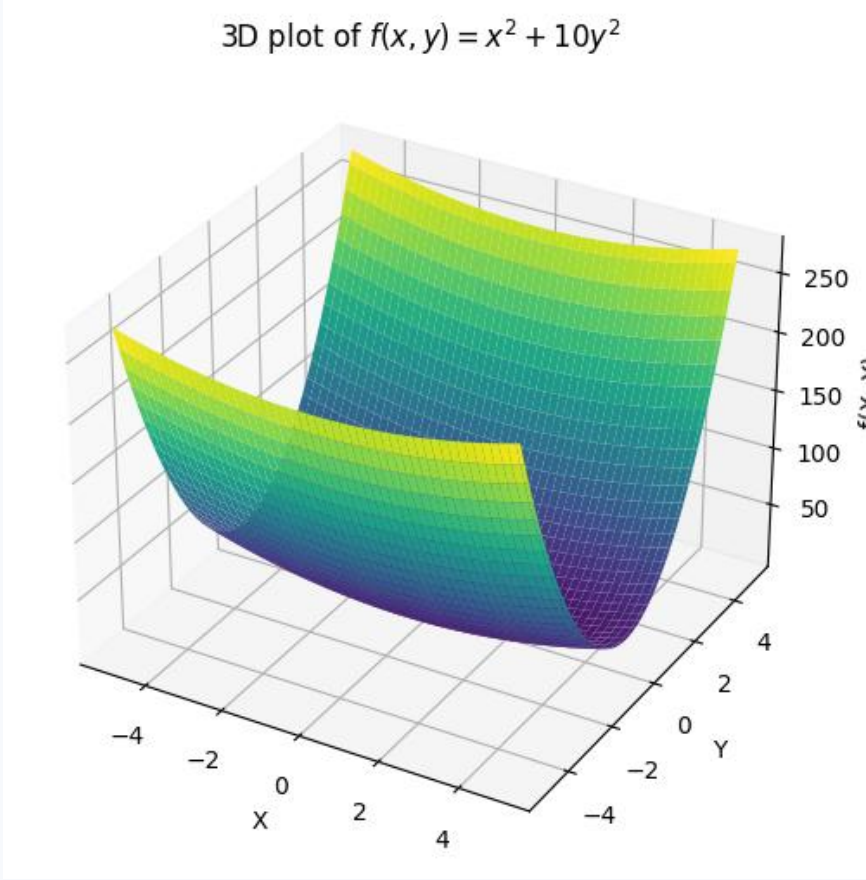
NEBIUS ACADEMY

Why gradient optimization is tricky

Badly conditioned problems

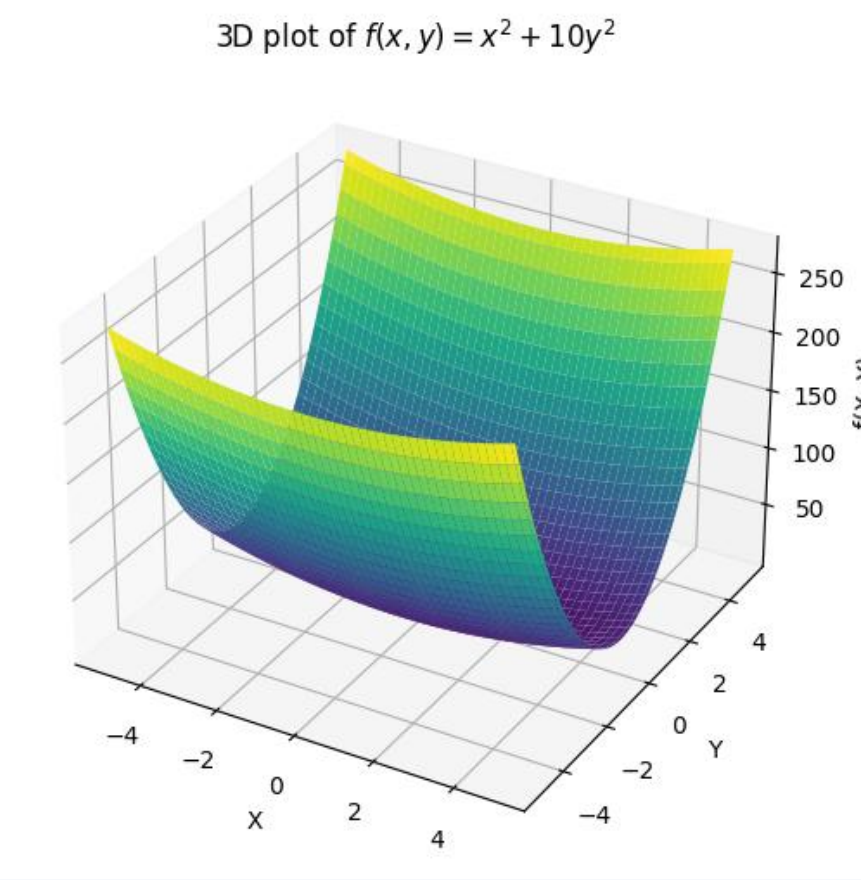
Very steep gradients are bad





Having slopes of very different magnitude along different directions is bad

Badly conditioned problems



Very steep gradients are bad

A bit of math:

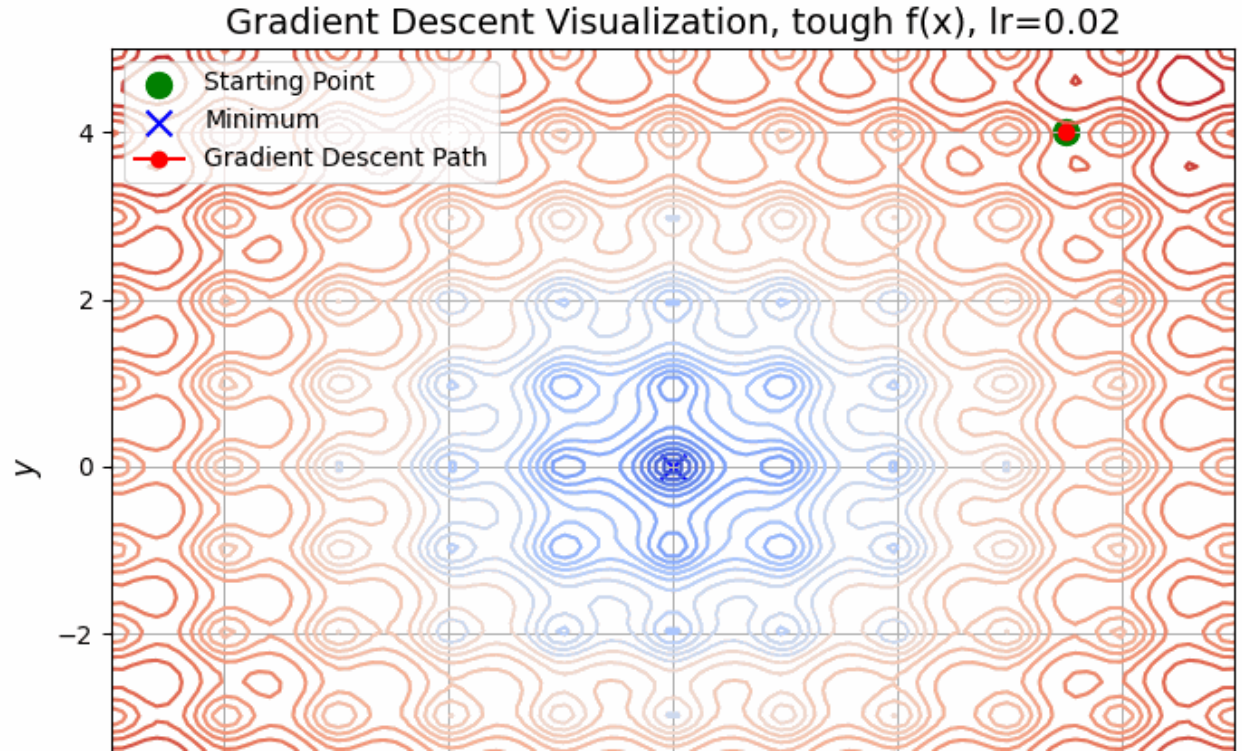
- For a smooth and strongly convex f after k steps the precision ($|f(x_k) - f(x_*)|$) will be

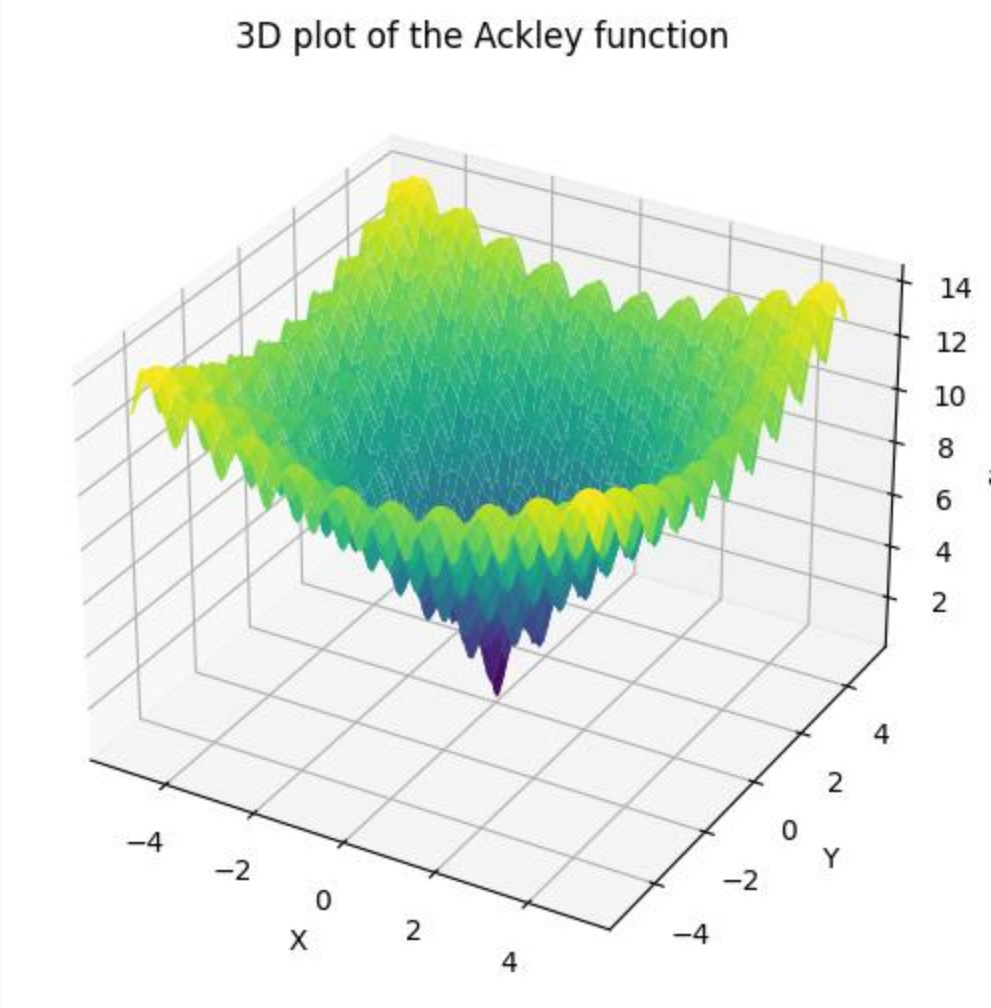
$$O\left(\min\left\{R^2 \exp\left(-\frac{k}{4\kappa}\right), \frac{R^2}{k}\right\}\right)$$

where $R^2 = |x_0 - x_*|$ and κ is the **condition number**.

Local minima

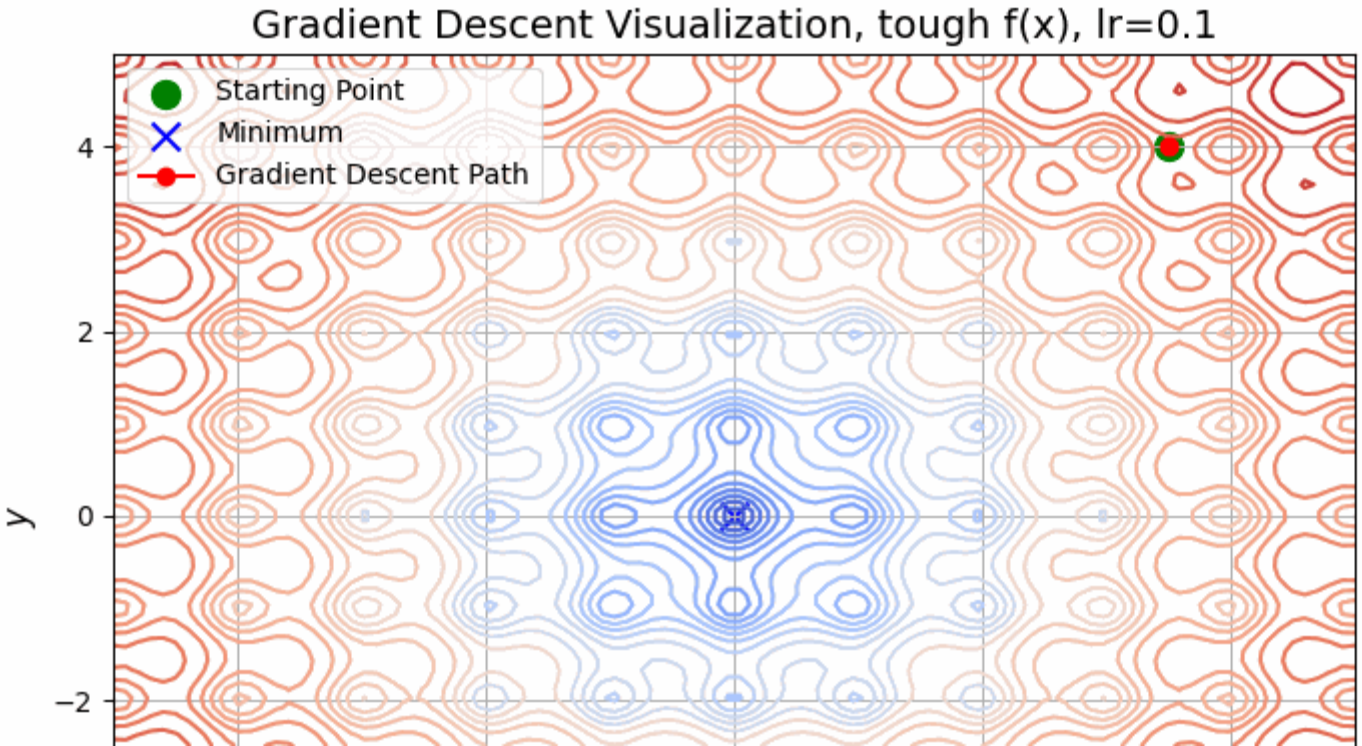
Gradient descent may get stuck in local optima

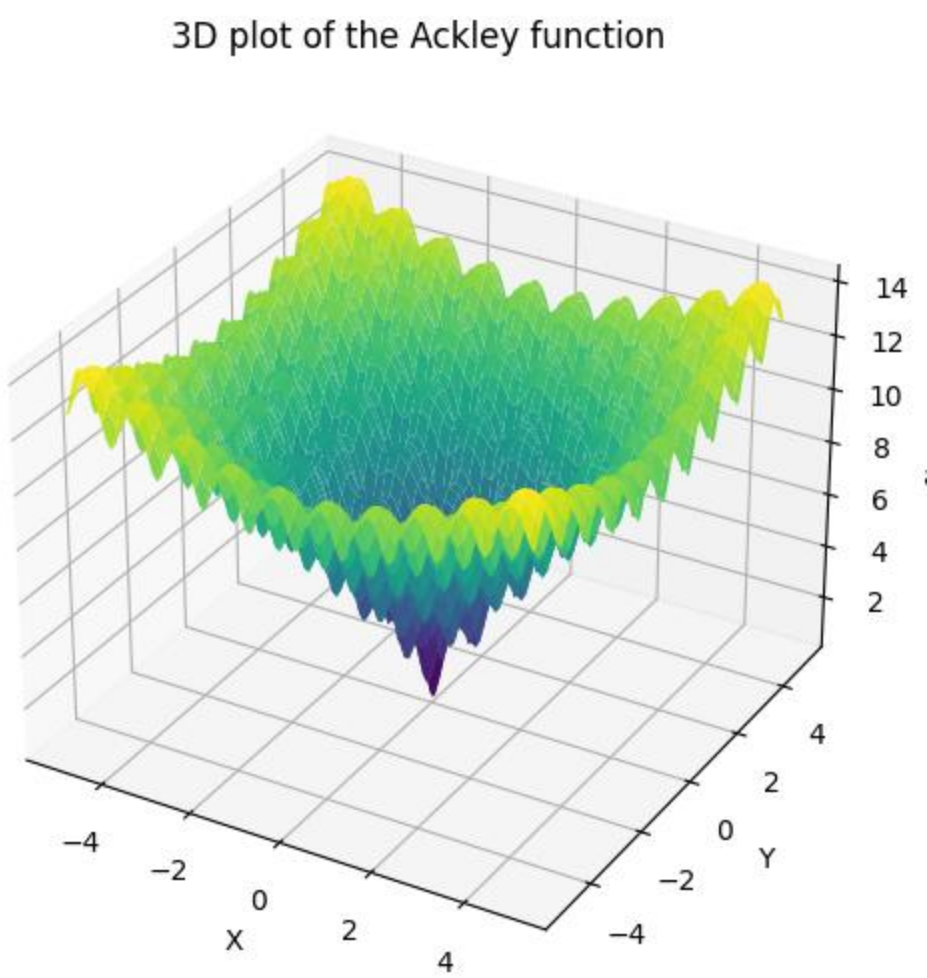




Local minima

Gradient descent may get stuck in local optima





Gradient descent in Machine Learning

$$\mathcal{L}(y, X, w) = \frac{1}{N}\sum_{i=1}^{N}\mathcal{L}(y_i, x_i, w) \quad \rightsquigarrow$$

$$\nabla_w \mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\nabla_w \mathcal{L}(y_i, x_i, w) = \frac{1}{N}\left(-2y^T X + 2wX^T X\right)$$

See any problems?

Gradient descent in Machine Learning

Why gradient descent isn't used:

- Avoid manifesting large matrices in the memory
- If your dataset is huge, we don't feedback from the system anytime soon.

Stochastic gradient descent

Stochastic gradient descent

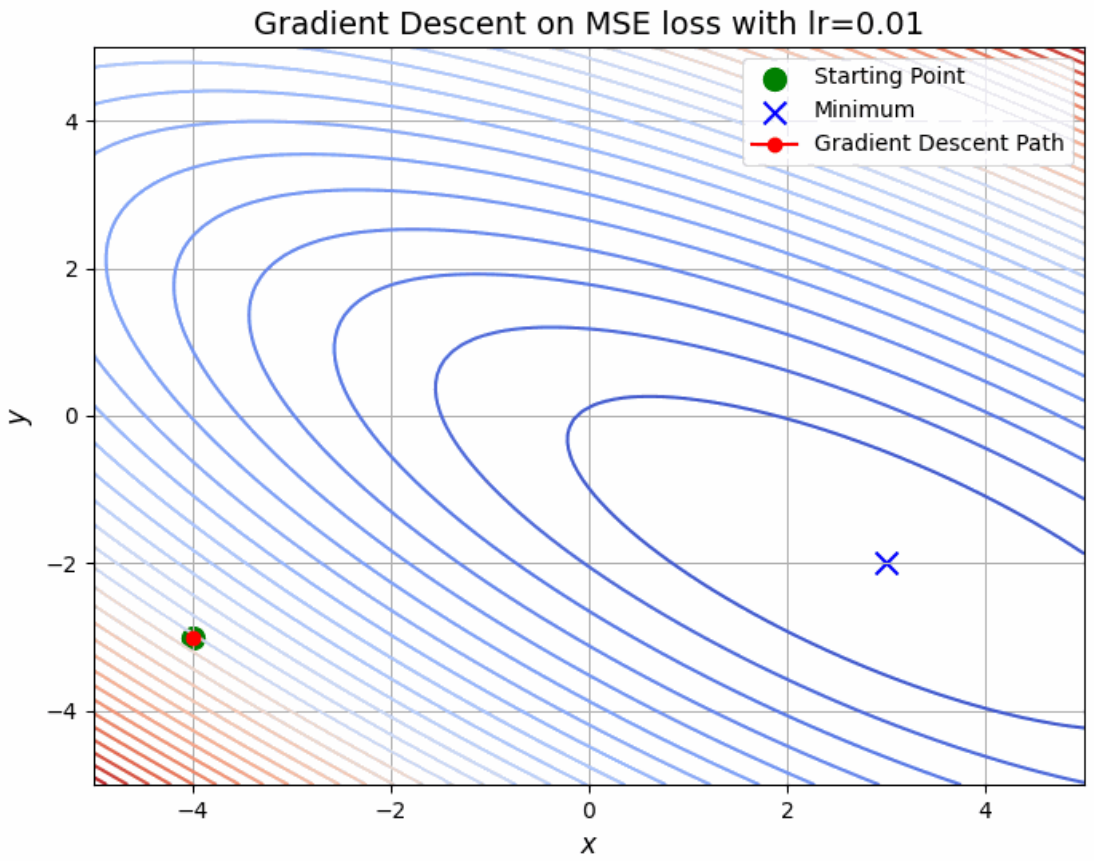
GD	**SGD with batch size B**
$w_{k+1} = w_k - \alpha \sum_{i=1}^{N} \nabla_w \mathcal{L}(y_i, x_i, w_k)$	Pick B **random** data points $\left(x_{i_s}, y_{i_s}\right)_{s=1}^{B}$, $w_{k+1} = w_k - \alpha \sum_{s=1}^{B} \nabla_w \mathcal{L}\left(y_{i_s}, x_{i_s}, w_k\right)$

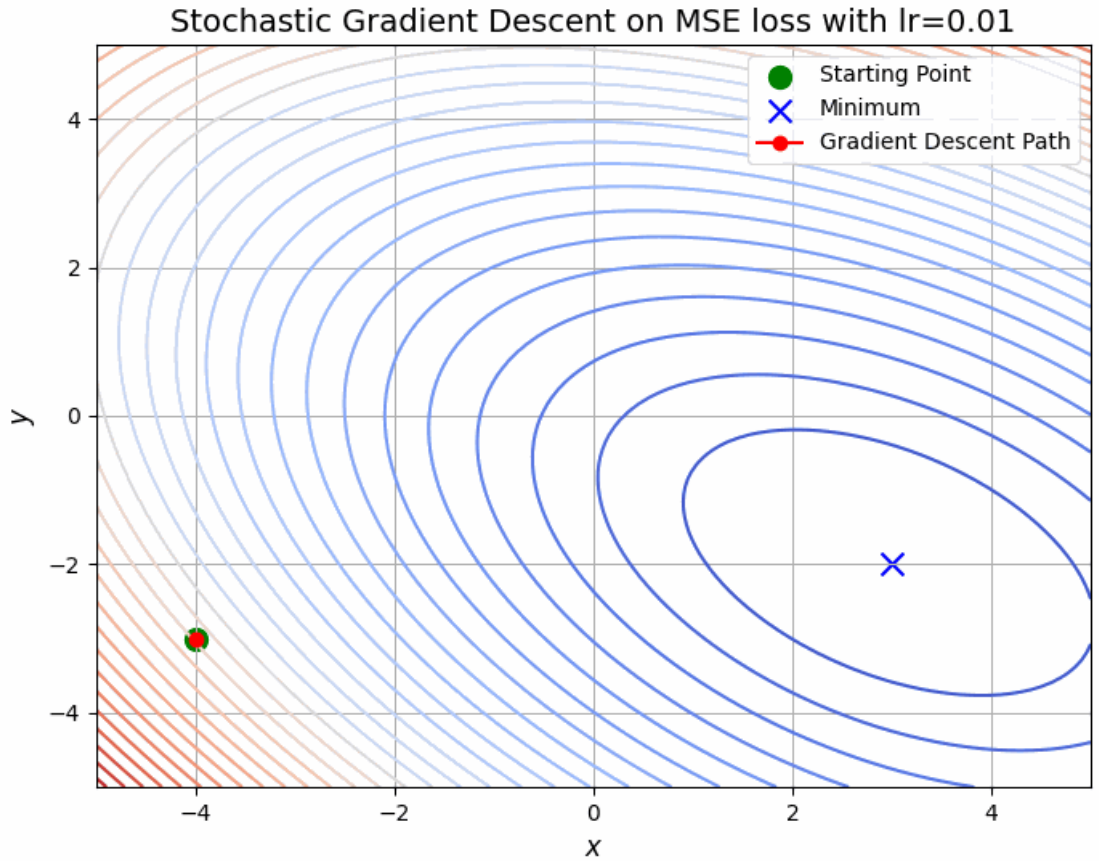
A typical implementation

GD	**SGD with batch size B**
$w_{k+1} = w_k - \alpha \sum_{i=1}^{N} \nabla_w \mathcal{L}(y_i, x_i, w_k)$	For several **epochs**: • Randomize data order • Now, for t=0..(N/B-1) $w_{k+1} = w_k - \alpha \sum_{i=Bt}^{Bt+B-1} \nabla_w \mathcal{L}(y_i, x_i, w_i)$

Comparing behaviour: GD vs SGD

SGD is erratic, and the smaller the batch size, the worse





A tricky question

GD	**SGD with batch size B**
$w_{k+1} = w_k - \frac{\alpha}{N} \sum_{i=1}^{N} \nabla_w \mathcal{L}(y_i, x_i, w_k)$	Pick B **random** data points $\left(x_{i_s}, y_{i_s}\right)_{s=1}^{B}$, $w_{k+1} = w_k - \frac{\alpha}{???} \sum_{s=1}^{B} \nabla_w \mathcal{L}\left(y_{i_s}, x_{i_s}, w_k\right)$

A tricky question

GD	**SGD with batch size B**
$w_{k+1} = w_k - \frac{\alpha}{N}\sum_{i=1}^{N} \nabla_w \mathcal{L}(y_i, x_i, w_k)$	Pick B **random** data points $(x_{i_s}, y_{i_s})_{s=1}^{B}$, $w_{k+1} = w_k - \frac{\alpha}{???}\sum_{s=1}^{B} \nabla_w \mathcal{L}(y_{i_s}, x_{i_s}, w_k)$
$\nabla_w \mathcal{L}(y, X, w) = \frac{1}{N}\sum_{i=1}^{N} \nabla_w \mathcal{L}(y_i, x_i, w_k)$	$\nabla_w \mathcal{L}(y, X, w) \approx \frac{1}{B}\sum_{s=1}^{B} \nabla_w \mathcal{L}(y_{i_s}, x_{i_s}, w_k)$ A Monte Carlo estimate

A tricky question

GD	**SGD with batch size B**
$w_{k+1} = w_k - \frac{\alpha}{N}\sum_{i=1}^{N} \nabla_w \mathcal{L}(y_i, x_i, w_k)$	Pick B **random** data points $(x_{i_s}, y_{i_s})_{s=1}^{B}$, $w_{k+1} = w_k - \frac{\alpha}{\boldsymbol{B}}\sum_{s=1}^{B} \nabla_w \mathcal{L}(y_{i_s}, x_{i_s}, w_k)$
$\nabla_w \mathcal{L}(y, X, w) = \frac{1}{N}\sum_{i=1}^{N} \nabla_w \mathcal{L}(y_i, x_i, w_k)$	$\nabla_w \mathcal{L}(y, X, w) \approx \frac{1}{B}\sum_{s=1}^{B} \nabla_w \mathcal{L}(y_{i_s}, x_{i_s}, w_k)$ A Monte Carlo estimate

NEBIUS ACADEMY

Modifications of SGD

(Stochastic) Gradient Descent

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$w_k = w_{k-1} - \alpha_k \cdot g_k$$

Momentum

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

Accumulates previous directions
Makes GD less erratic

$$w_k = w_{k-1} - \alpha_k \cdot \mu_k$$

AdaGrad

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$w_k = w_{k-1} - \alpha_k \cdot \frac{g_k}{\sqrt{g_k^2 + \epsilon}}$$

Adaptive step
for each coordinate
(~second-order methods)

RMSProp

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\nu_k = \beta_2 \cdot \nu_{k-1} + (1 - \beta_2) g_k^2$$

Adaptive step
for each coordinate
(~second-order methods)

$$w_k = w_{k-1} - \alpha_k \cdot \frac{g_k}{\sqrt{\nu_k + \epsilon}}$$

Bringing everything together: almost Adam

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

Accumulates previous directions
Makes GD less erratic

$$\nu_k = \beta_2 \cdot \nu_{k-1} + (1 - \beta_2) g_k^2$$

Adaptive step
for each coordinate
(~second-order methods)

$$w_k = w_{k-1} - \alpha_k \cdot \frac{\mu_k}{\sqrt{\hat{\nu}_k + \epsilon}}$$

Bringing everything together: Adam

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

Accumulates previous directions
Makes GD less erratic

$$\nu_k = \beta_2 \cdot \nu_{k-1} + (1 - \beta_2) g_k^2$$

Adaptive step
for each coordinate
(~second-order methods)

$$\hat{\mu}_k = \frac{\mu_k}{1 - \beta_1^k}, \qquad \hat{\nu}_k = \frac{\nu_k}{1 - \beta_2^k}$$

Bias correction

$$w_k = w_{k-1} - \alpha_k \cdot \frac{\hat{\mu}_k}{\sqrt{\hat{\nu}_k + \epsilon}}$$

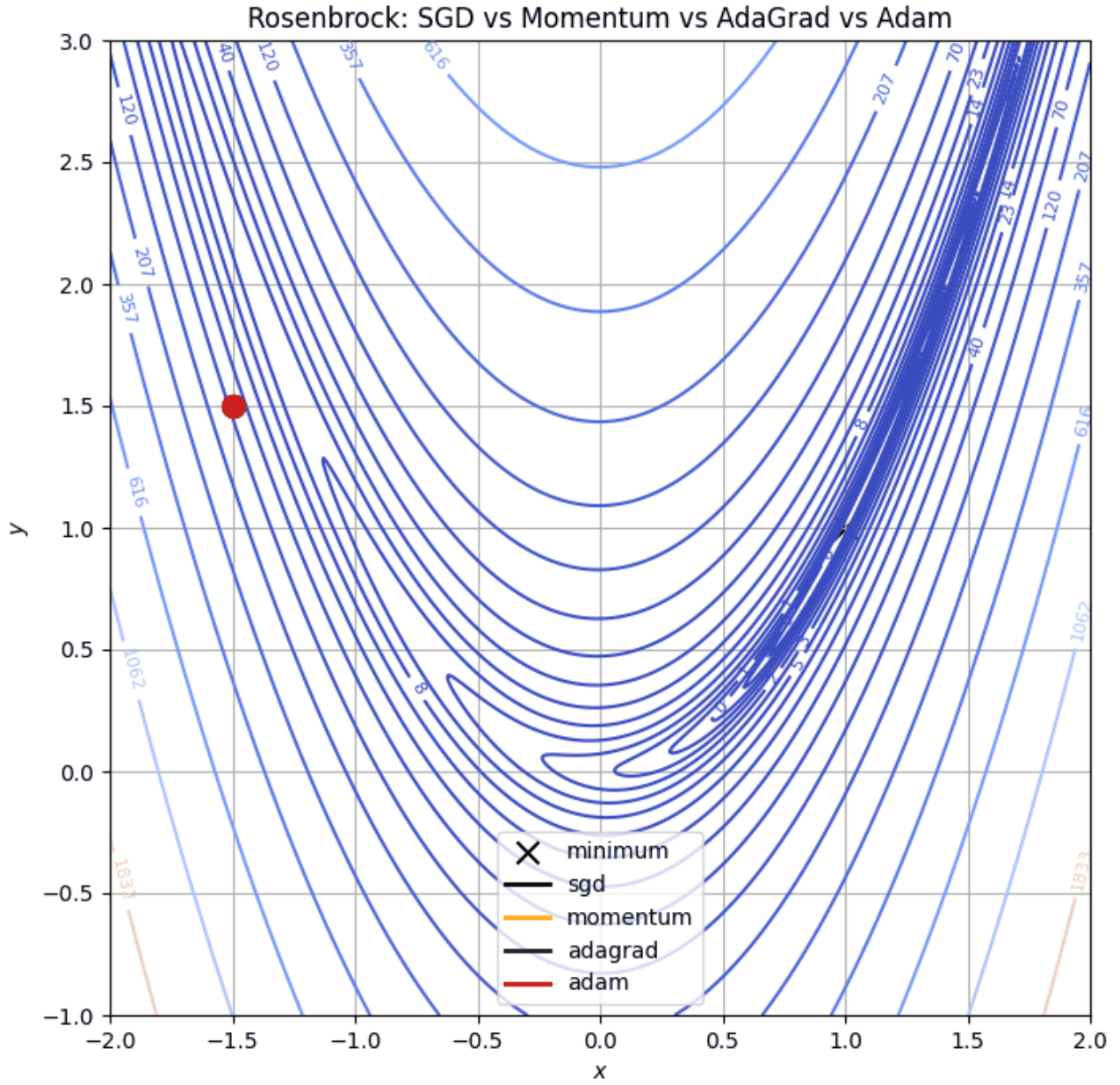
A simple experiment

Rosenbrock function

$$(1 - x)^2 + 100(y - x^2)^2$$

Has global minimum in

$(1, 1)$

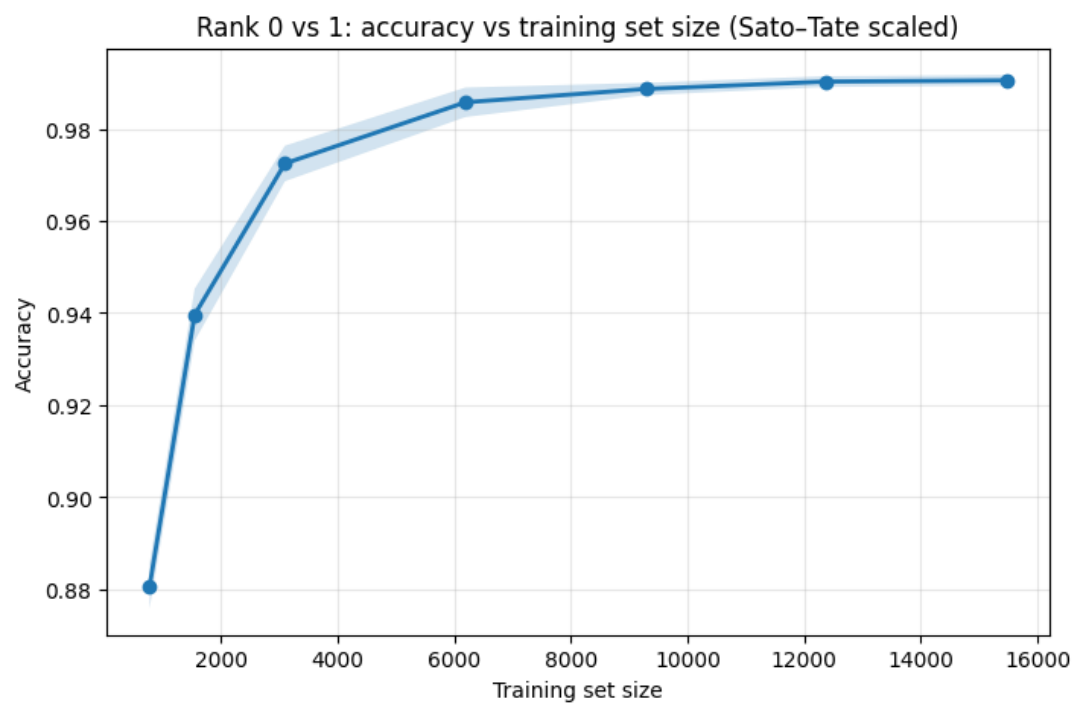


NEBIUS ACADEMY

Pathology of ML pipelines

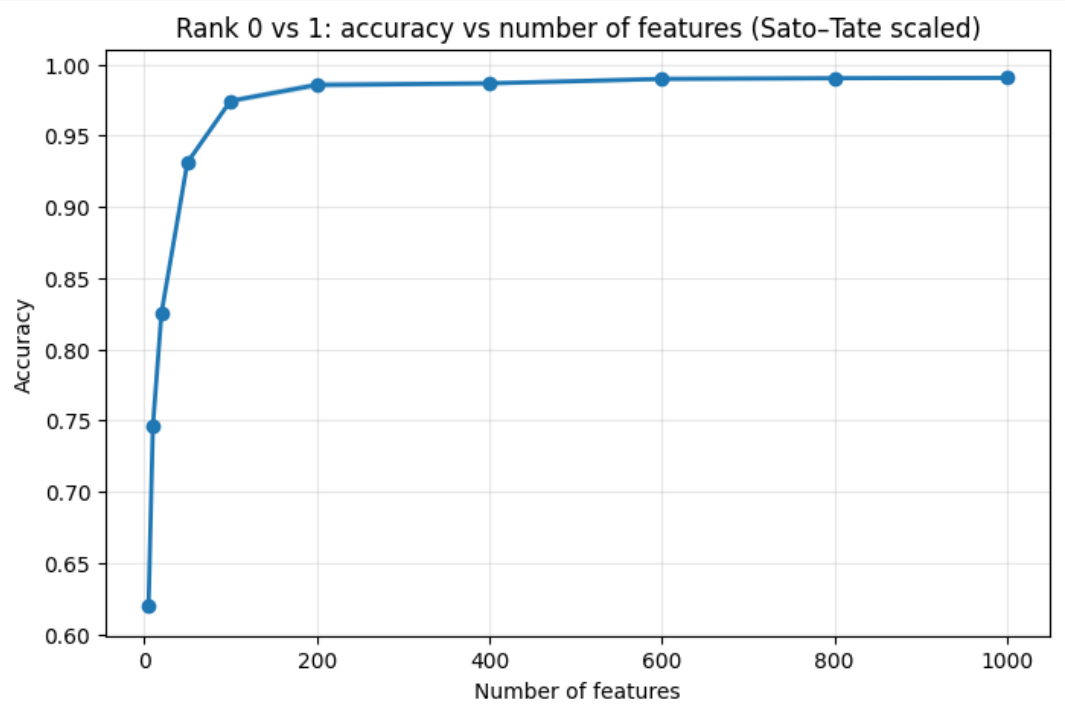
Not enough data

- Medical data
- Low-resource language translation in pre-LLM era



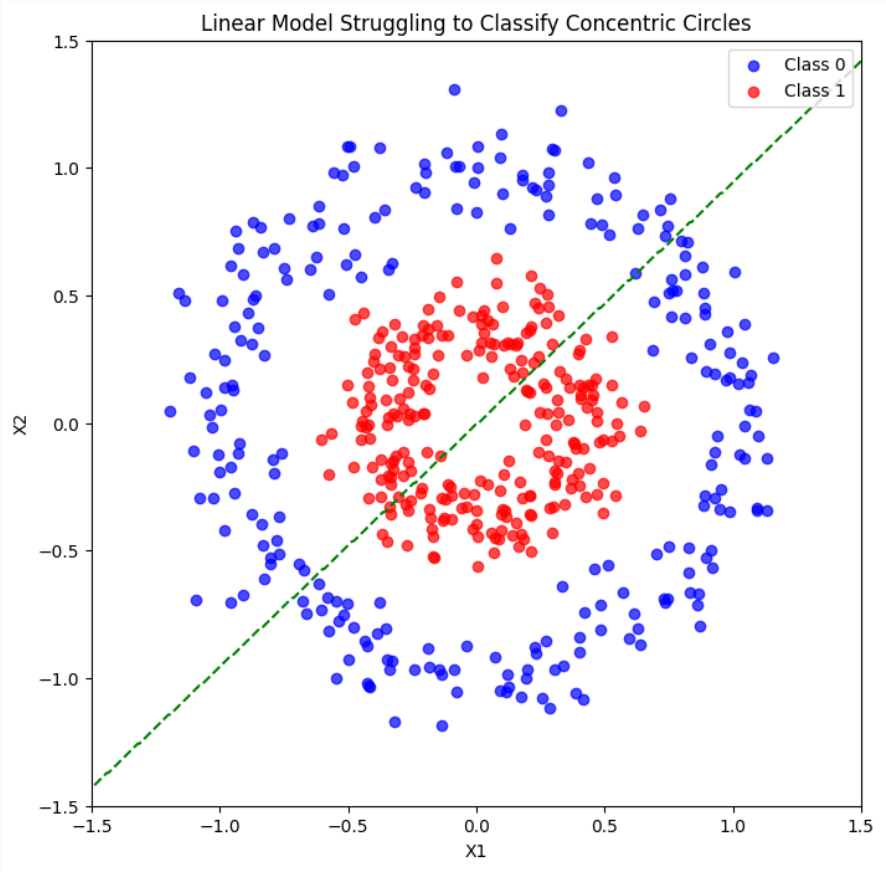
Not enough features

- Medical data: don’t know much about the patient’s previous life
- Self-driving cars with cameras only (no LiDAR)



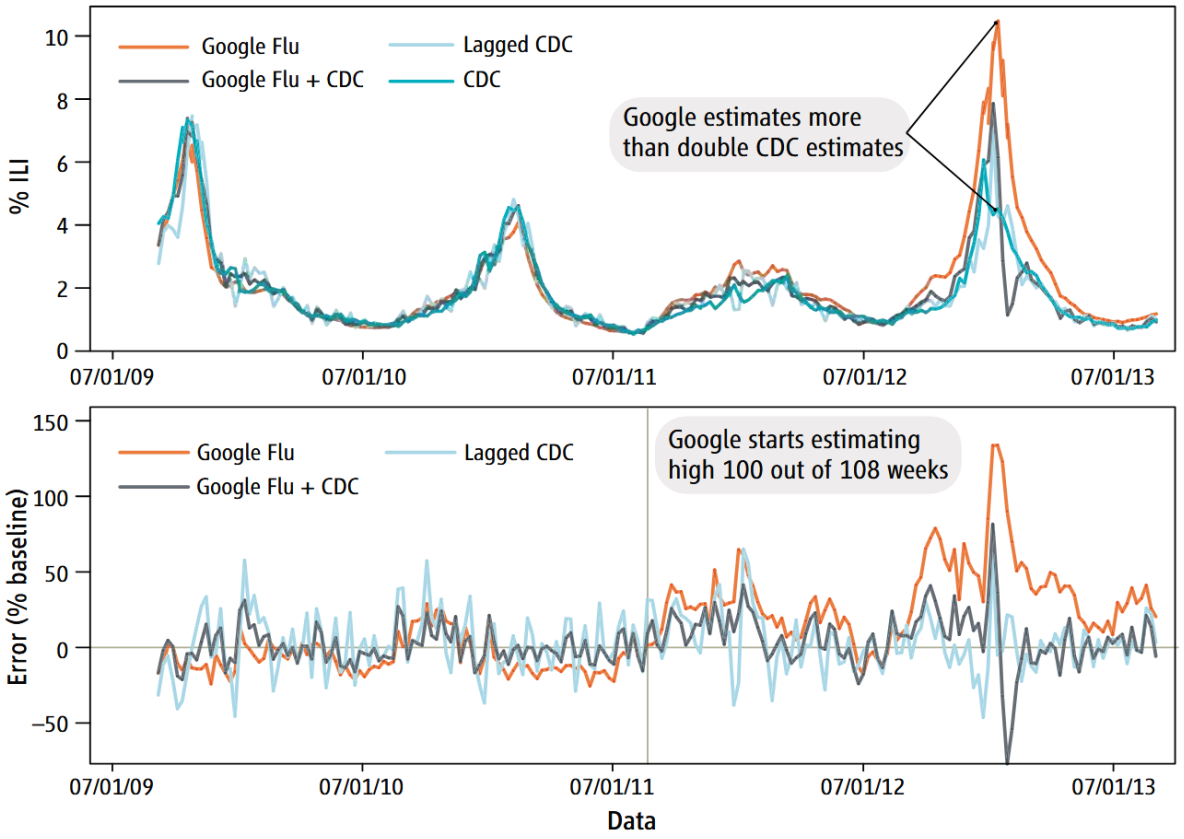
Features/models not expressive enough

- Text generation before LLMs
- Self-driving cars with cameras only (no LiDAR) ?



Spurious correlations

- Google Flu Trends



GFT overestimation. GFT overestimated the prevalence of flu in the 2012–2013 season and overshot the actual level in 2011–2012 by more than 50%. From 21 August 2011 to 1 September 2013, GFT reported overly high flu prevalence 100 out of 108 weeks. (**Top**) Estimates of doctor visits for ILI. "Lagged CDC" incorporates 52-week seasonality variables with lagged CDC data. "Google Flu + CDC" combines GFT, lagged CDC estimates, lagged error of GFT estimates, and 52-week seasonality variables. (**Bottom**) Error [as a percentage {[Non-CDC estmate)−(CDC estimate)]/(CDC) estimate)]. Both alternative models have much less error than GFT alone. Mean absolute error (MAE) during the out-of-sample period is 0.486 for GFT, 0.311 for lagged CDC, and 0.232 for combined GFT and CDC. All of these differences are statistically significant at $P < 0.05$. See SM.

https://gking.harvard.edu/sites/g/files/omnuum7116/files/gking/files/0314policyforumff.pdf

Training on the test data

- LLM leaderboards
- Benchmarks leak
- Sometimes we want to optimize a metric too much…

The Leaderboard Illusion https://arxiv.org/abs/2504.20879

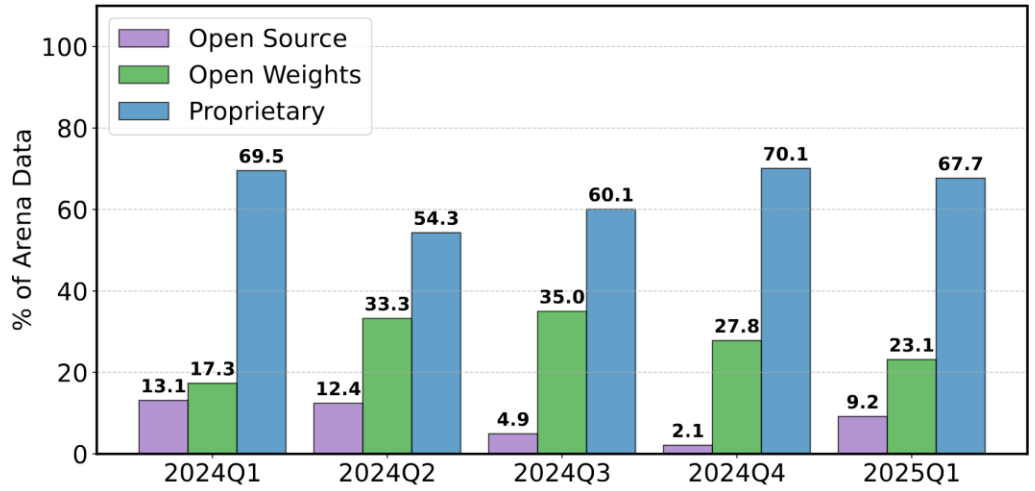
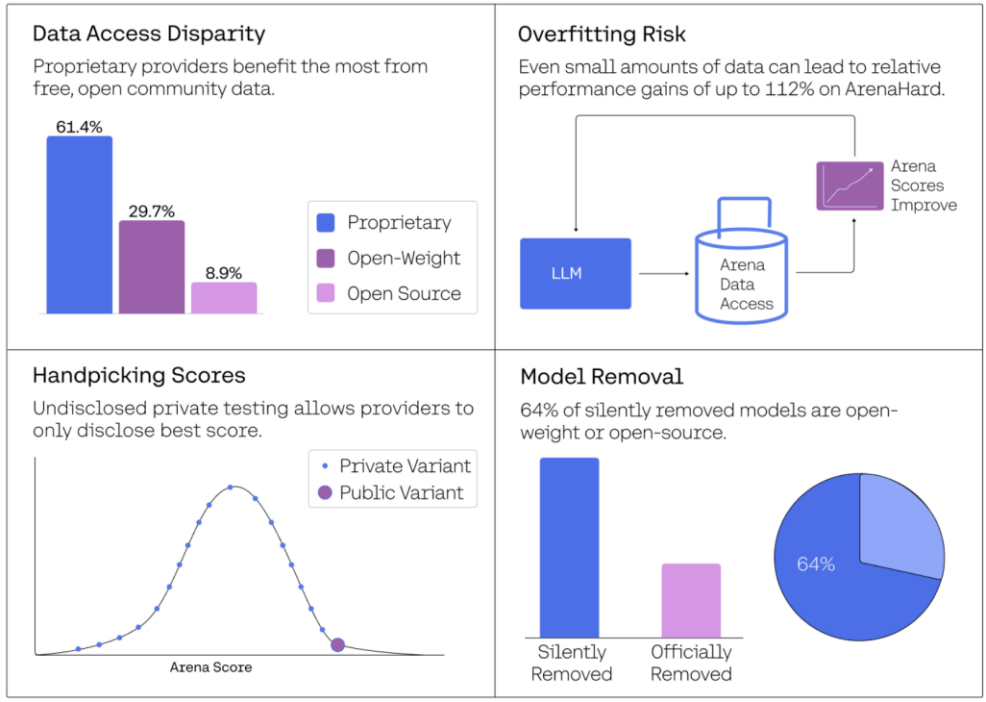
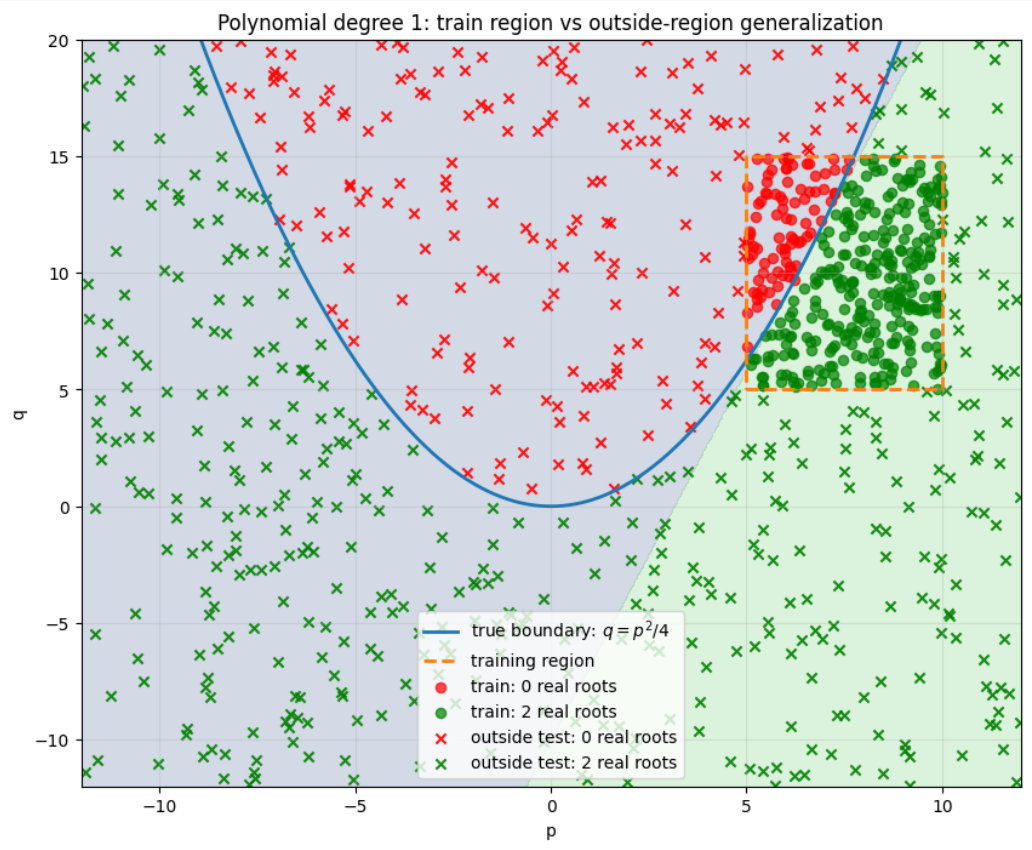


Figure 3: **Volume of Arena battles involving proprietary, open-weight, and fully open-source model providers from January 2024 to March 2025, based on `leaderboard-stats`.** Proprietary models consistently received the largest share of data—ranging from 54.3% to 70.1%. Open-weight and fully open-source models receive significantly less data, in some cases receiving less than half the amount of data as proprietary developers. This imbalance in data access exacerbates the performance gap, reinforcing unequal access to high-quality in-distribution data.



Training data distribution != test data distribution

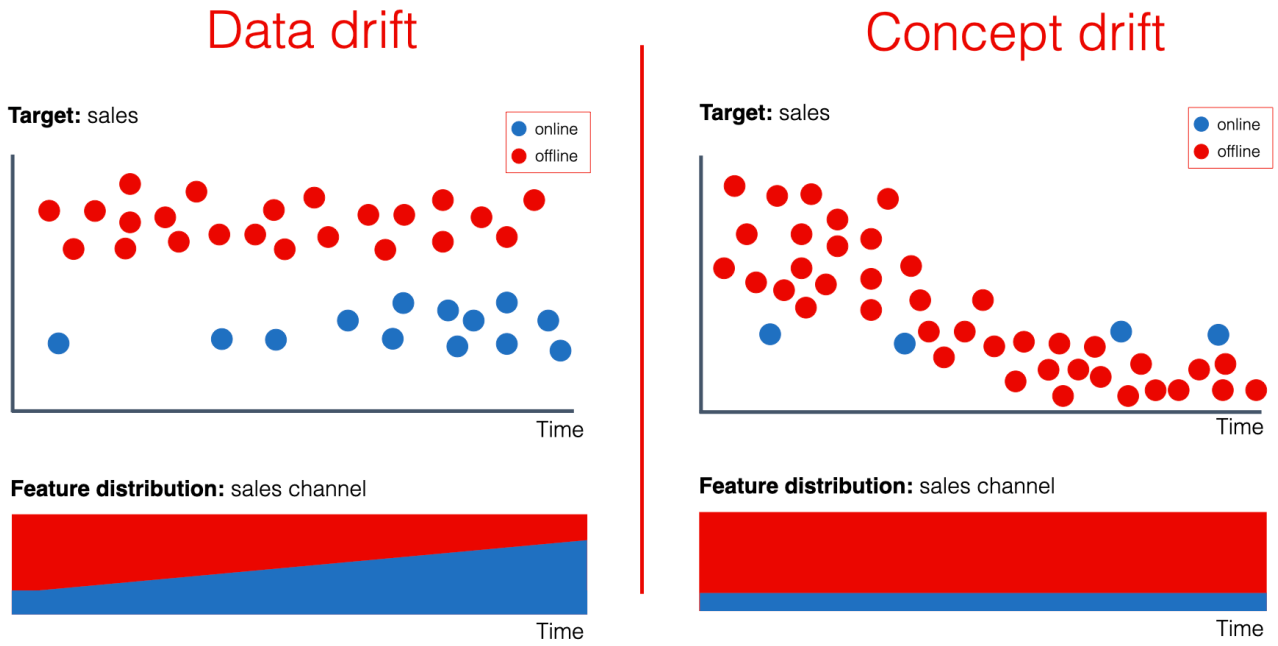
- Oncology Expert Advisor (trained on clean, synthetic records)
- Self-driving cars in winter :D



Data drifts

- Apple’s privacy changes hurting ability of other companies to target and measure digital advertising

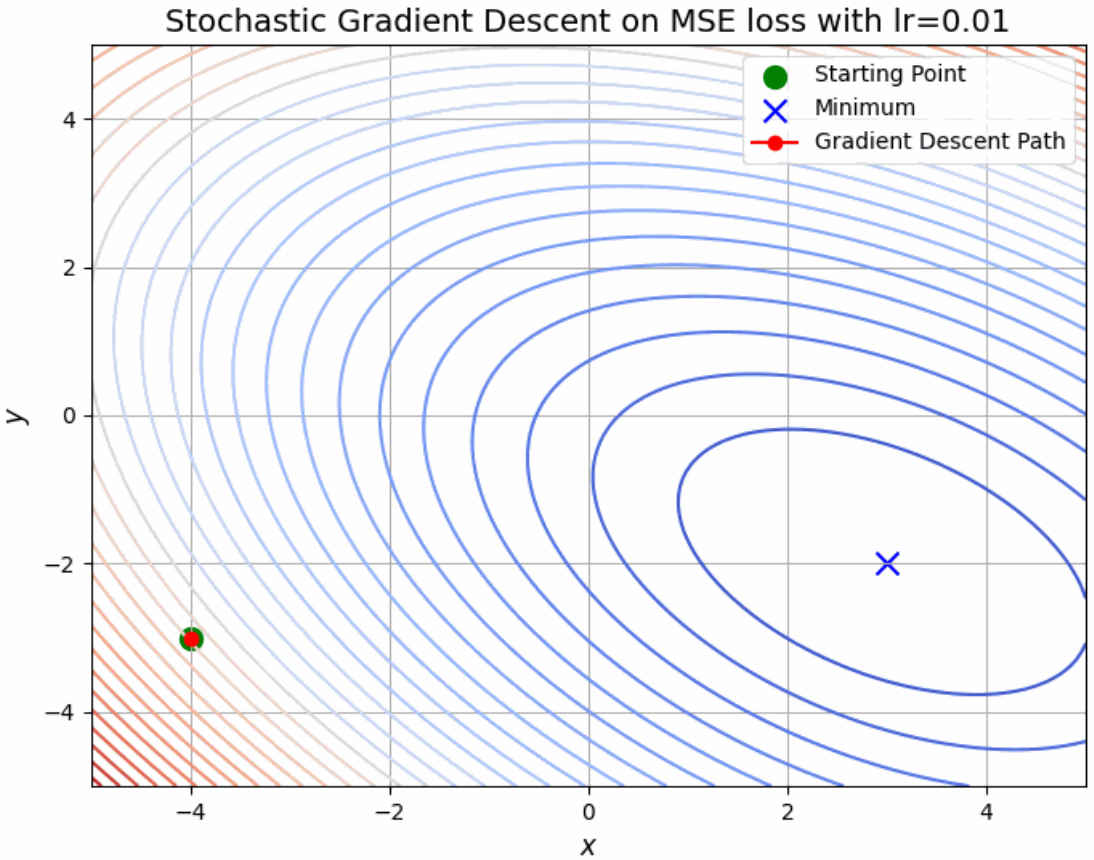
- Customers’ habits change



https://www.evidentlyai.com/ml-in-production/data-drift

The training process goes awry

- We've already seen this ☺



NEBIUS ACADEMY

Evaluation

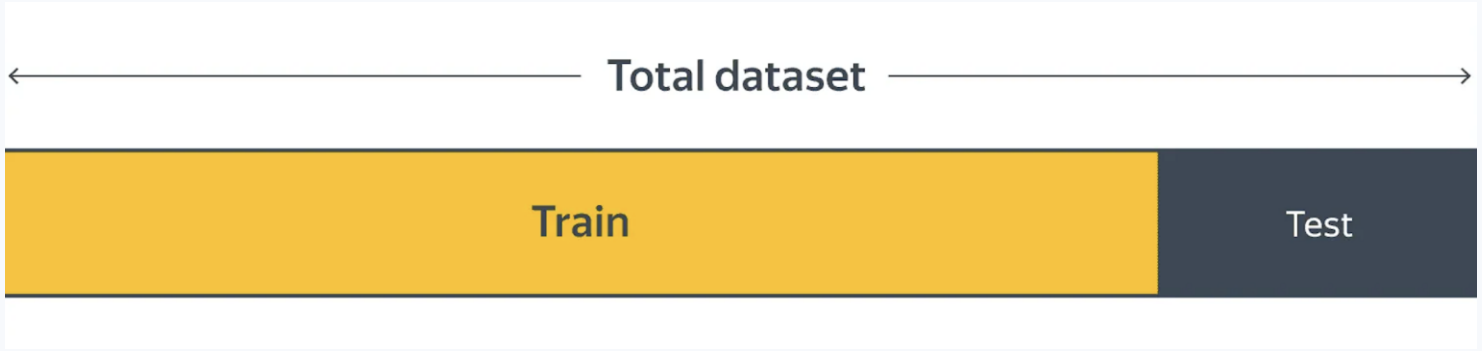
Train/test split

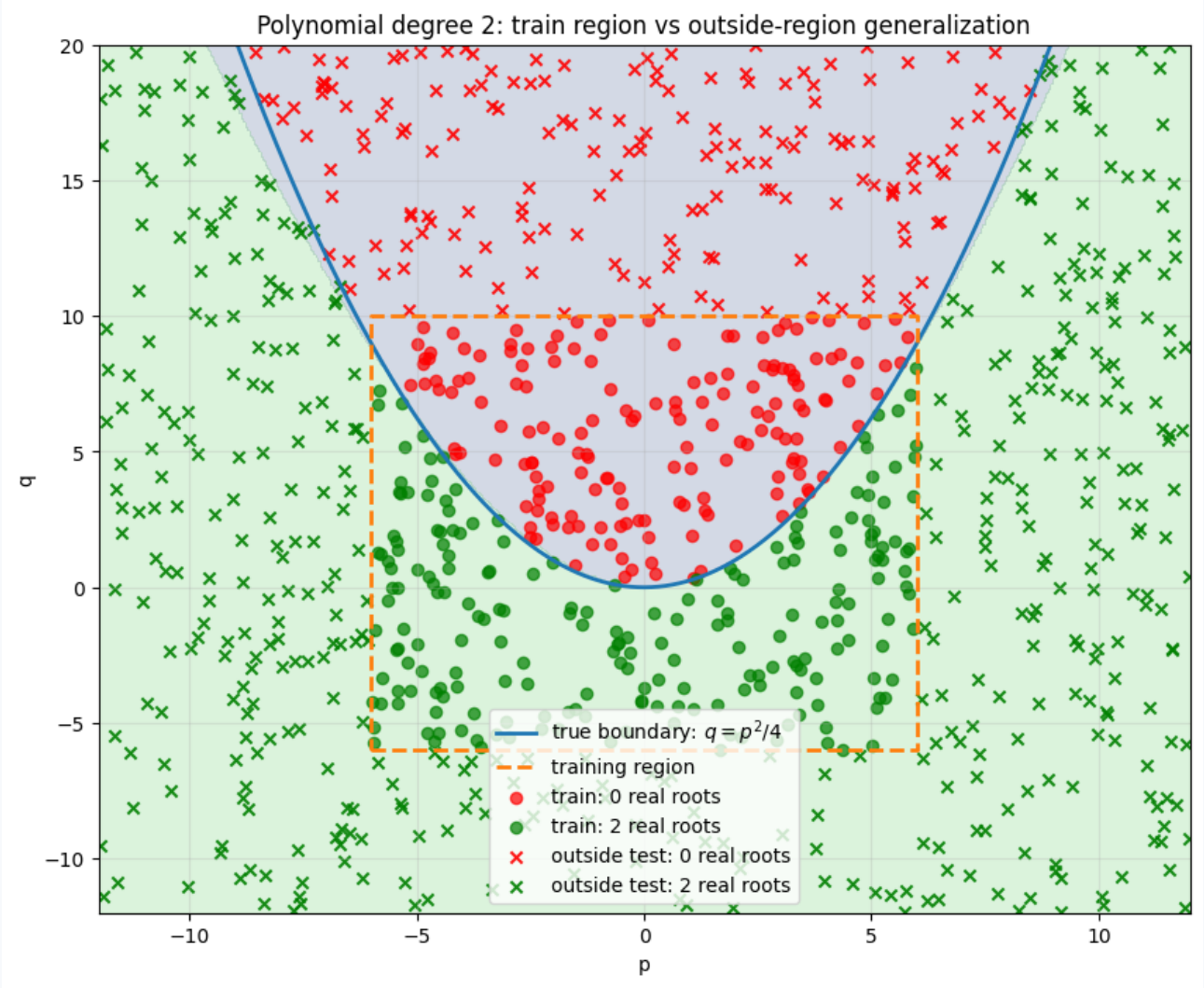
Train set – used to train the model

Test set – used to evaluate the model

Split them randomly; fix random seed; stratify by **target**

Train and test set should be from the same data distribution




Polynomial degree 2: train region vs outside-region generalization
20
15
10
5
0
−5
−10
q
−10
−5
0
5
10
p
true boundary: $q = p^2/4$
training region
train: 0 real roots
train: 2 real roots
outside test: 0 real roots
outside test: 2 real roots

Never allow test leak into train!

A typical example of how it happens:

You have a pipeline:

1. Data normalization: $x_i \mapsto \frac{x_i - \mu(x)}{\sigma(x)}$

2. Linear classifier

You compute $\mu(x)$ and $\sigma(x)$ on the **whole data**.

This is wrong! You should compute them only on the training part

NEBIUS ACADEMY

Metrics for binary classification

Is accuracy a good metric?

How to understand if we're doing well?

$$\text{accuracy} = \frac{\text{Right guesses}}{\text{Total data}}$$

Is accuracy a good metric?

$$\text{accuracy} = \frac{\text{Right guesses}}{\text{Total patients}}$$

Example (Imbalanced classes):

Common cold, 1000

A nasty virus, 20

Is accuracy a good metric?

$$\text{accuracy} = \frac{\text{Right guesses}}{\text{Total patients}}$$

Example (Imbalanced classes):

Common cold, 1000

A nasty virus, 20

Classifying them all with common cold gives 98% accuracy

How to choose a metric?

The main question: what is our priority?

Examples:

1. We don’t want to miss a single person infected with the nasty virus

2. We don’t want to waste our time on treating people with common cold! If we diagnose someone with the nasty virus, we’d better be correct!

The magic square

Notation:

Class 1 = Nasty virus (positive)

Class 0 = Common cold (negative)

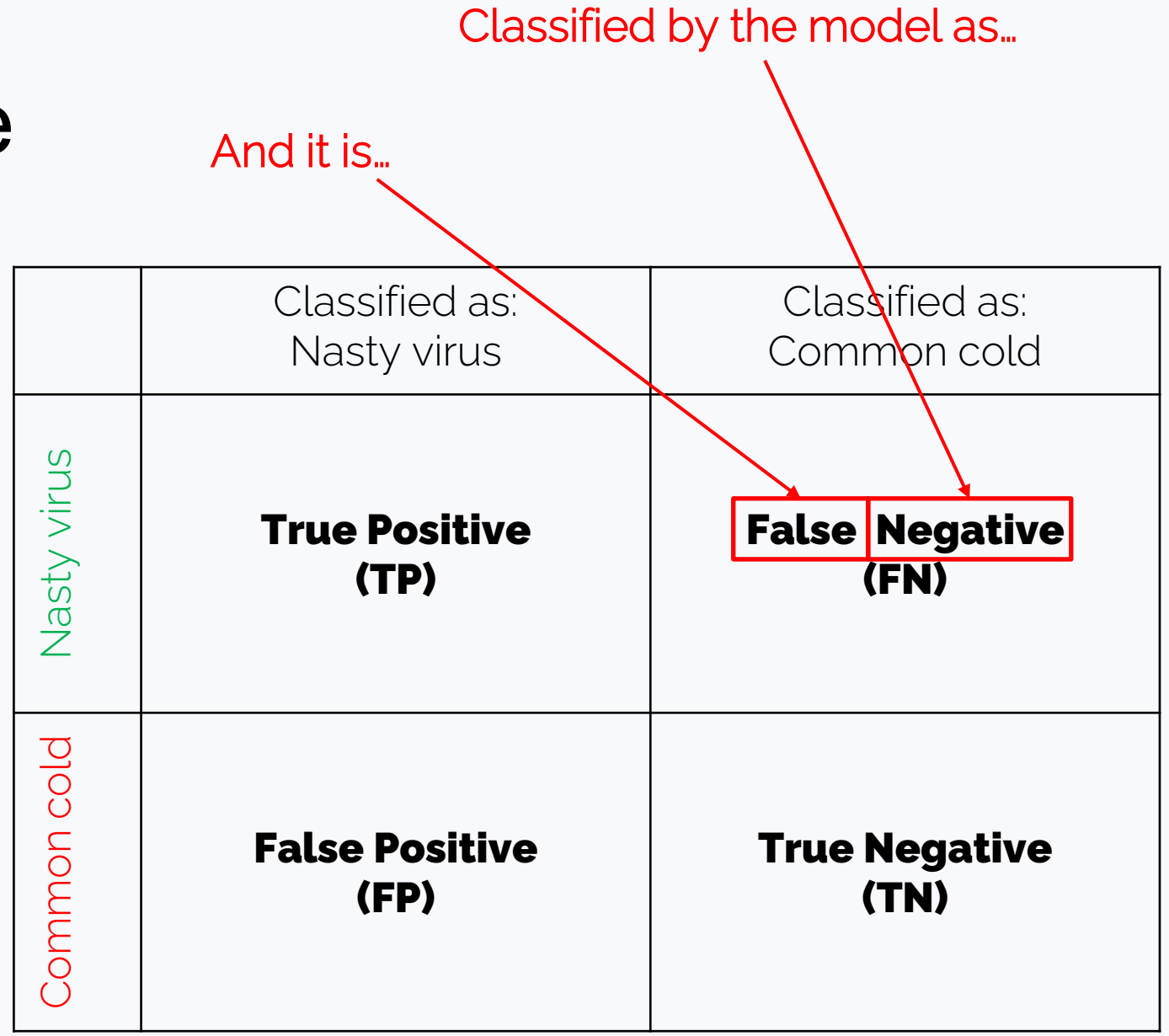
	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	**True Positive (TP)**	**False Negative (FN)**
Common cold	**False Positive (FP)**	**True Negative (TN)**

The magic square

Notation:

Class 1 = Nasty virus (positive)

Class 0 = Common cold (negative)



The magic square

Notation:

Class 1 = Nasty virus (positive)

Class 0 = Common cold (negative)

	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	**True Positive (TP)** **20**	**False Negative (FN)** **0**
Common cold	**False Positive (FP)** **0**	**True Negative (TN)** **1000**

No serious case missed!

More **positives** (nasty virus) should be **true positives** (classified with the nasty virus). So, we need:

$$\frac{TP}{TP + FN}$$

This metric is called **recall**

`	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	**True Positive (TP)**	**False Negative (FN)**
Common cold	**False Positive (FP)**	**True Negative (TN)**

No penny wasted!

More patients classified with the nasty should be **true positives** (really have the nasty virus). So, we need:

$$\frac{TP}{TP + FP}$$

This metric is called **precision**

`	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	**True Positive (TP)**	**False Negative (FN)**
Common cold	**False Positive (FP)**	**True Negative (TN)**

Hacking recall (both recall = 1)

$$\textbf{recall} = \frac{TP}{TP+FN}$$

`	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	**20**	**0**
Common cold	**1000**	**0**

`	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	**20**	**0**
Common cold	**0**	**1000**

Hacking precision (precision 1)

$$\textbf{precision} = \frac{TP}{TP+FP}$$

`	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	**1**	**19**
Common cold	**0**	**1000**

`	Classified as: Nasty virus	Classified as: Common cold
Nasty virus	**20**	**0**
Common cold	**0**	**1000**

Combining precision and recall

Maximize precision while keeping recall > 0.9

Or vice versa

An attempt at fusing precision and recall

F1 score is their harmonic mean:

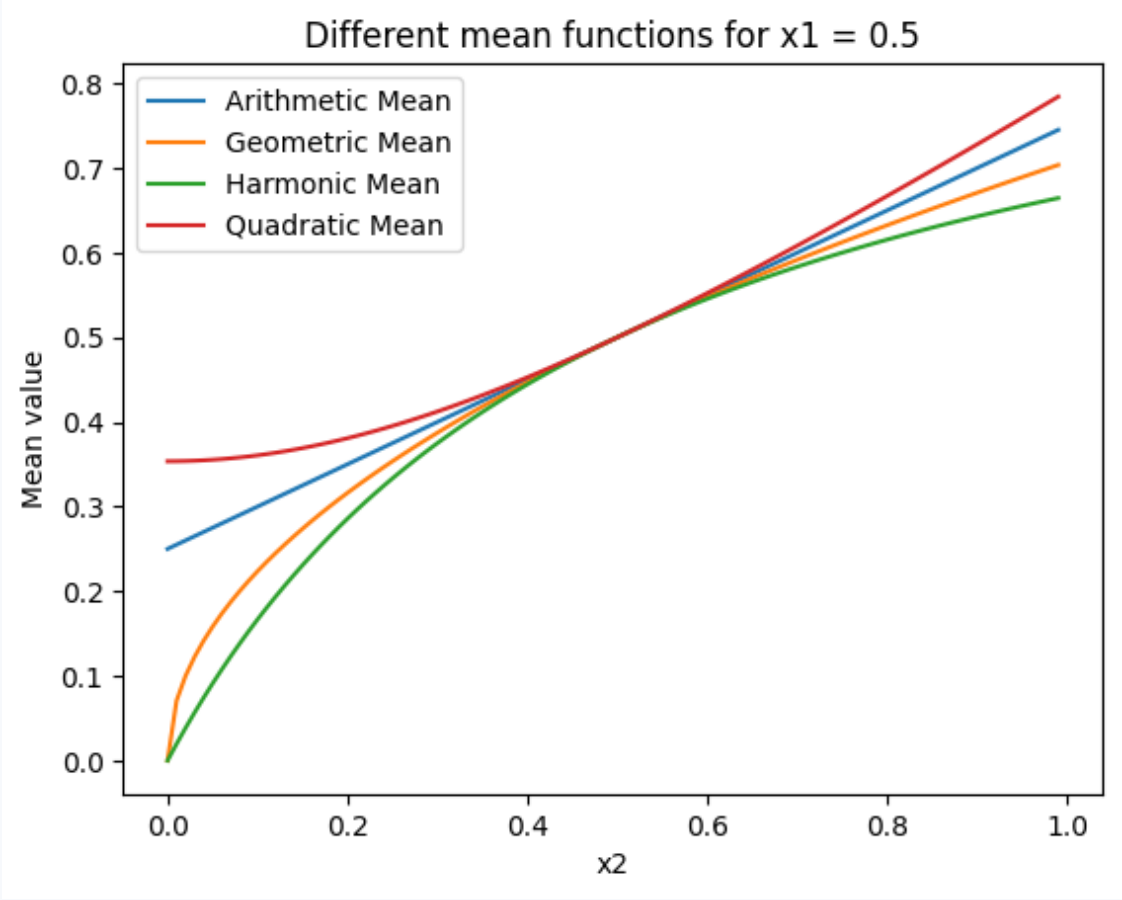
$$F1 = \frac{2 \cdot \text{ precision} \cdot \text{recall}}{\text{precision} + \text{ recall}}$$

Math:

$$\frac{1}{F1} = \frac{1}{2}\left(\frac{1}{\text{precision}} + \frac{1}{\text{recall}}\right)$$

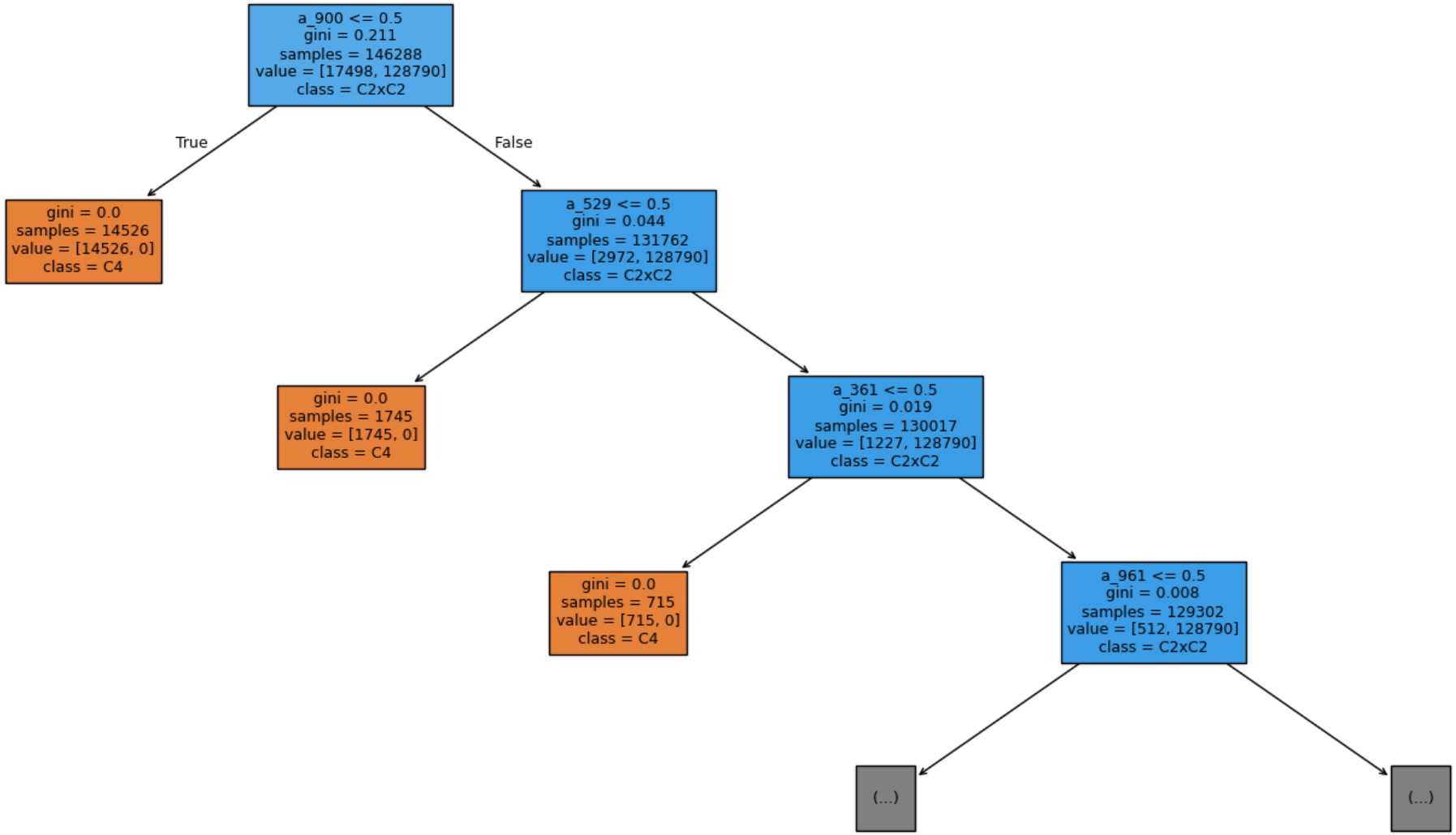
Among other means, it is the pessimistic one:

$$\frac{2ab}{a+b} \leq \sqrt{ab} \leq \frac{a+b}{2} \leq \sqrt{\frac{a^2+b^2}{2}}$$

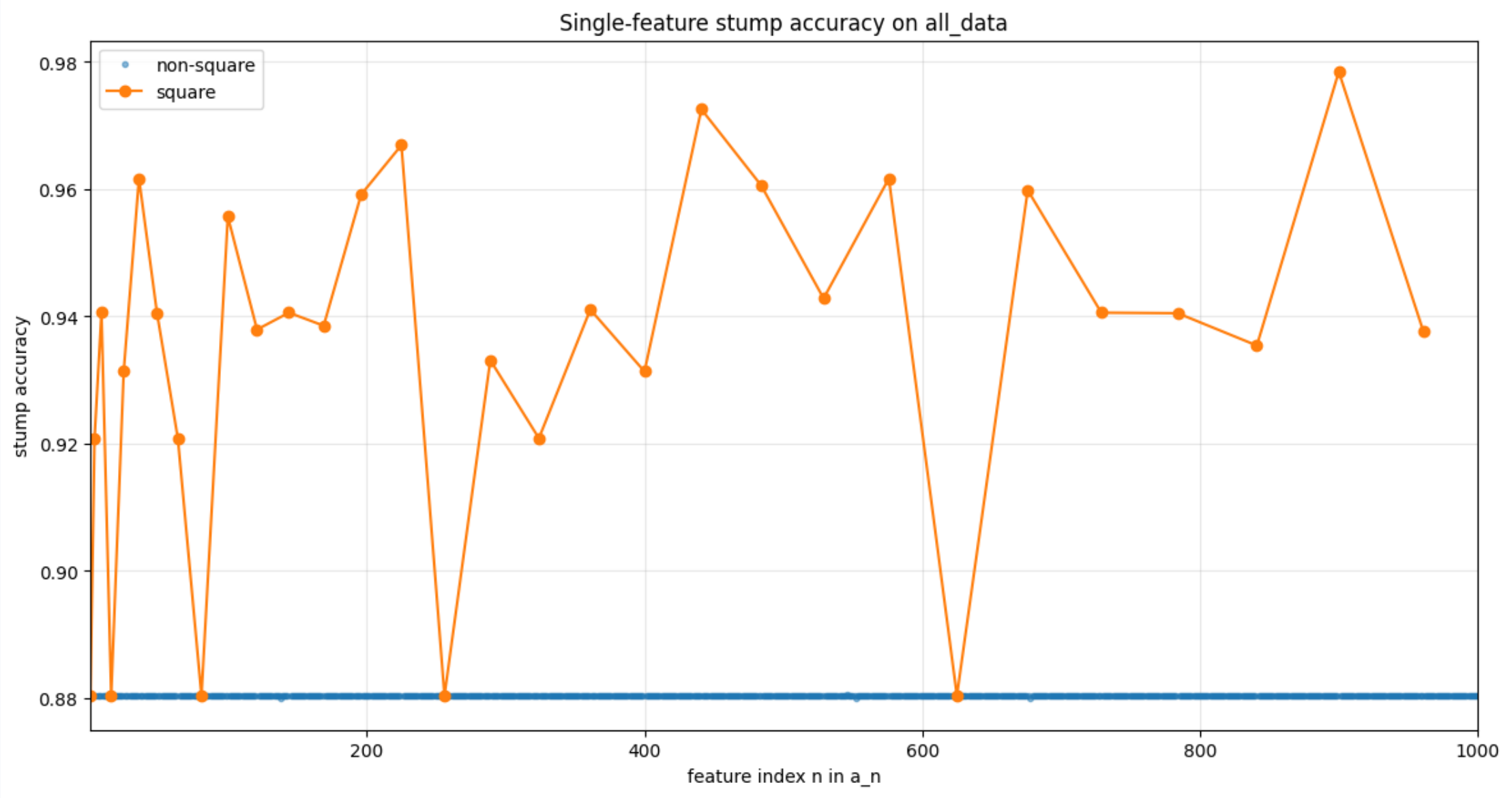


A cautionary tale

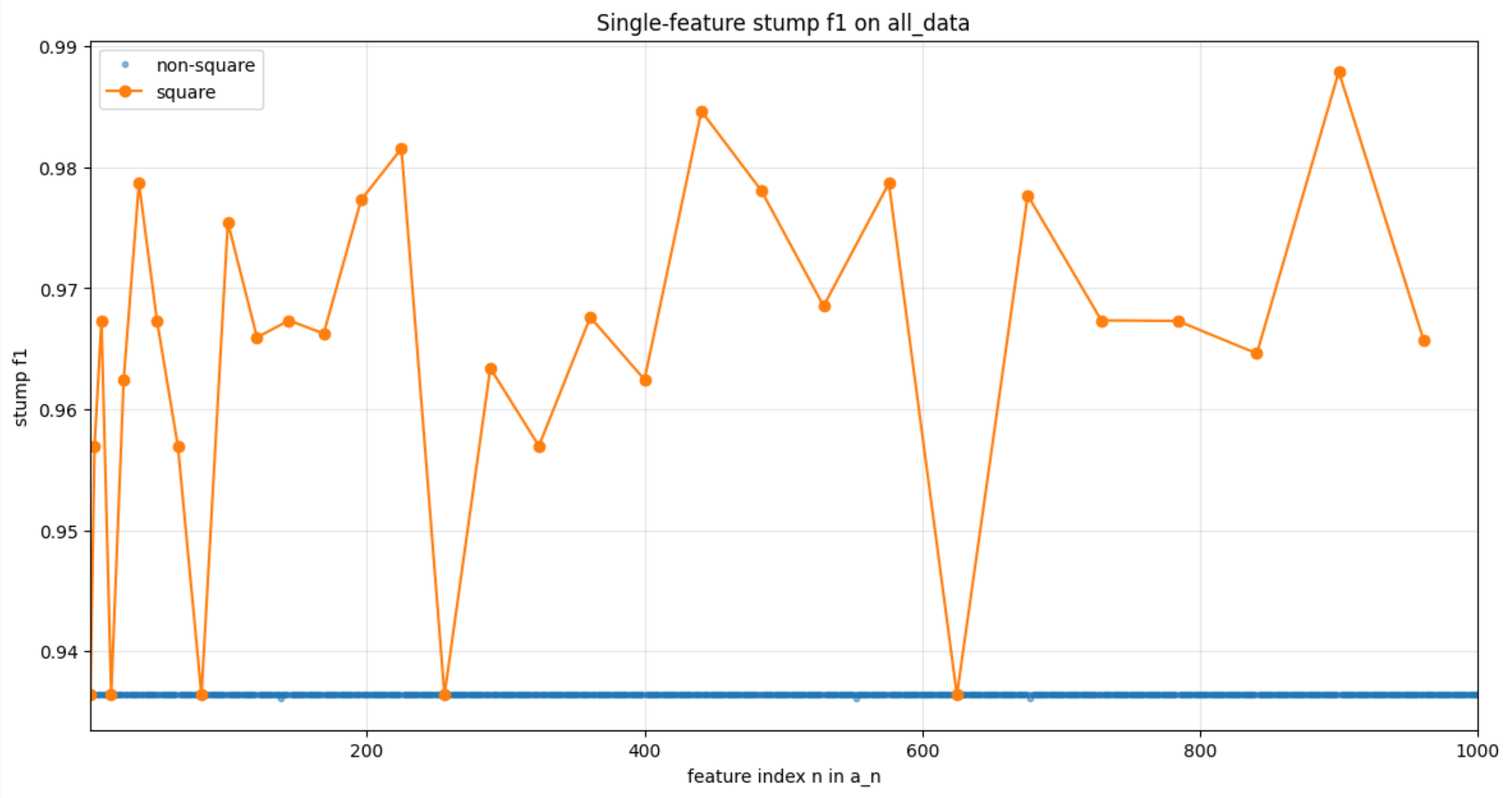
C_4 vs $C_2 \times C_2$



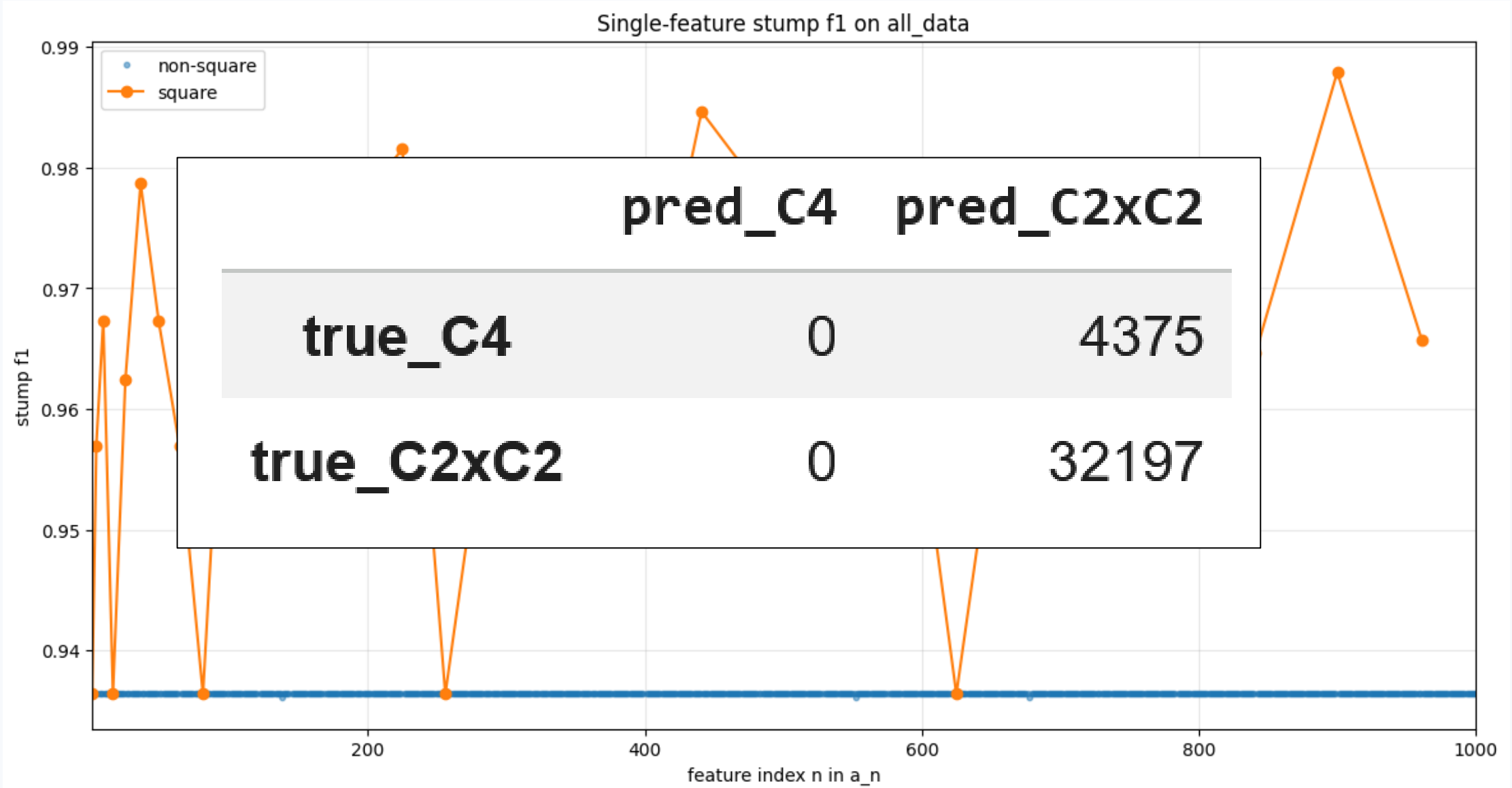
C_4 vs $C_2 \times C_2$



C_4 vs $C_2 \times C_2$



C_4 vs $C_2 \times C_2$



	pred_C4	pred_C2xC2
true_C4	0	4375
true_C2xC2	0	32197

NEBIUS ACADEMY

Regularization

What is regularization?

- Countering overfitting
- Stabilizing optimization
- Imposing conditions

Multicollinearity

The worst regression task:

$$X = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \qquad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

$$\mathcal{L}(y, X, w) = \frac{1}{N}\sum_{i=1}^{N}(y_i - w_1)^2$$

Solution:

$$w_1 = \text{mean}(y_i), \qquad w_2 - \text{any!}$$

Multicollinearity

$$X^T X = \begin{pmatrix} N & 0 \\ 0 & 0 \end{pmatrix}$$

The worst regression task:

Not invertible!

$$X = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \qquad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

$$\hat{w}^T = \boldsymbol{(X^T X)^{-1}} X^T y$$

$$\mathcal{L}(y, X, w) = \frac{1}{N} \sum_{i=1}^{N} (y_i - w_1)^2$$

Solution:

$$w_1 = \text{mean}(y_i), \qquad w_2 - \text{any!}$$

Multicollinearity

A quite bad regression task:

$$X = \begin{pmatrix} 1 & \varepsilon \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \qquad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

ε is a very small number.

$$\widehat{w}^T = \left(\boldsymbol{X^T X}\right)^{-1} X^T y$$

$$X^T X = \begin{pmatrix} N & \varepsilon \\ \varepsilon & \varepsilon^2 \end{pmatrix}$$

is close to degenerate.

Multicollinearity

A quite bad regression task:

$$X = \begin{pmatrix} 1 & \varepsilon \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \qquad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

ε is a very small number.

$$\widehat{w}^T = \boldsymbol{(X^T X)^{-1}} X^T y$$

$$X^T X = \begin{pmatrix} N & \varepsilon \\ \varepsilon & \varepsilon^2 \end{pmatrix}$$

is close to degenerate.

$$(X^T X)^{-1} = \frac{1}{N-1} \begin{pmatrix} 1 & -\frac{1}{\varepsilon} \\ -\frac{1}{\varepsilon} & \frac{N}{\varepsilon^2} \end{pmatrix}$$

$$(X^T X)^{-1} X^T = \frac{1}{N-1} \begin{pmatrix} 0 & 1 & \cdots \\ \frac{N-1}{\varepsilon} & -\frac{1}{\varepsilon} & \cdots \end{pmatrix}$$

$$\widehat{w}^T \to (X^T X)^{-1} X^T (y + \delta)$$

A small change in y may lead to a catastrophic change in $\widehat{w}$.

Multicollinearity

A quite bad regression task:

$$X = \begin{pmatrix} 1 & \varepsilon \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \qquad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

$\varepsilon = 10^{-4}$,
$N = 100$

$$X^T X = \begin{pmatrix} 100 & 10^{-4} \\ 10^{-4} & 10^{-8} \end{pmatrix}$$

is close to degenerate.

$$(X^T X)^{-1} = \frac{1}{99}\begin{pmatrix} 1 & -10^4 \\ -10^4 & 10^{10} \end{pmatrix} \approx \begin{pmatrix} 10^{-2} & -10^2 \\ -10^2 & 10^8 \end{pmatrix}$$

$$(X^T X)^{-1} X^T \approx \begin{pmatrix} 0 & 10^{-2} & \cdots \\ 10^4 & 10^2 & \cdots \end{pmatrix}$$

$$\widehat{w}^T \to \begin{pmatrix} 0 & 10^{-2} & \cdots \\ 10^4 & 10^2 & \cdots \end{pmatrix}(y + 10^{-2})$$

A small change in y may lead to a catastrophic change in $\widehat{w}$.

Multicollinearity

A quite bad regression task:

$$X = \begin{pmatrix} 1 & \varepsilon \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{pmatrix}, \qquad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

$\varepsilon = 10^{-4}$,
$N = 100$

$$\widehat{w}^T = \left(\boldsymbol{X^T X}\right)^{-1} X^T y$$

$$X^T X + 10^{-4} I = \begin{pmatrix} 100 + 10^{-4} & 10^{-4} \\ 10^{-4} & 10^{-8} + 10^{-4} \end{pmatrix}$$

$$\left(X^T X + 10^{-4} I\right)^{-1} X^T$$
$$\approx \begin{pmatrix} 10^{-2} & 10^{-2} & \cdots \\ 1 & -10^2 & \cdots \end{pmatrix}$$

That's much better!

L2-regularization

How to fix an almost-degenerate matrix:

$$\hat{w}^T = \left(X^T X + \lambda I\right)^{-1} X^T y$$

L2-regularization

How to fix an almost-degenerate matrix:

$$\hat{w}^T = \left(X^T X + \lambda I\right)^{-1} X^T y$$

We can prove that this corresponds to the following task:

$$\mathcal{L}(y, X, w) = \left[\sum_{i=1}^{N} \left(y_i - x_i w^T\right)^2\right] + \lambda \|w\|_2^2$$

where $\|w\|_2^2 = w_1^2 + \cdots + w_D^2$

L2-regularization in general case

$$\mathcal{L}_{reg}(y, X, w) = \mathcal{L}(y, X, w) + \lambda \|w\|_2^2$$

where $\|w\|_2^2 = w_1^2 + \cdots + w_D^2$

Theory says that optimizing this new loss is equivalent to optimizing $\mathcal{L}(y, X, w)$ inside some $\{\|w\|_2 \leq R\}$

L2-regularization in general case

$$\mathcal{L}_{reg}(y, X, w) = \mathcal{L}(y, X, w) + \lambda \|w\|_2^2$$

where $\|w\|_2^2 = w_1^2 + \cdots + w_D^2$

Helps not to overfit on noisy features.

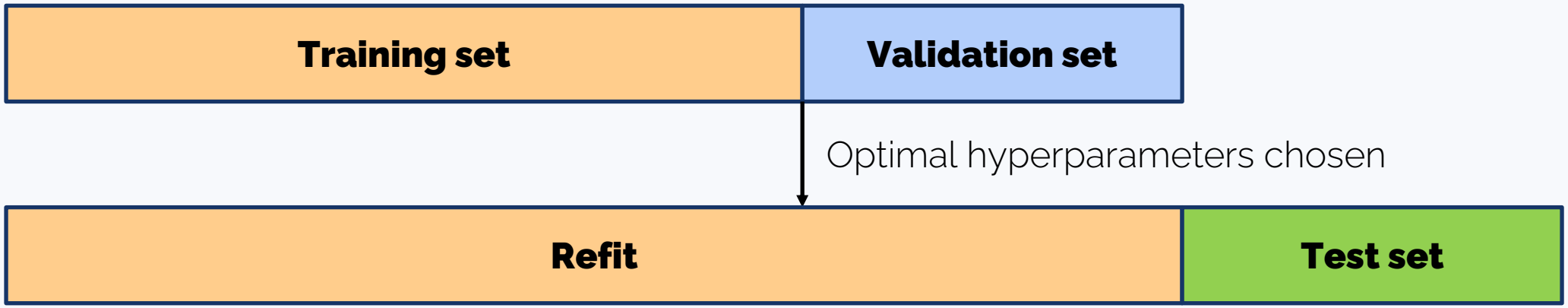
λ is a **hyperparameter**. It is tuned on a logarithmic scale.

Hyperparameter search

1. The **training set** is used to train the model.

2. The **validation set** is used to tune hyperparameters and select the best model.

3. The **test set** is used for a final evaluation of the model's performance

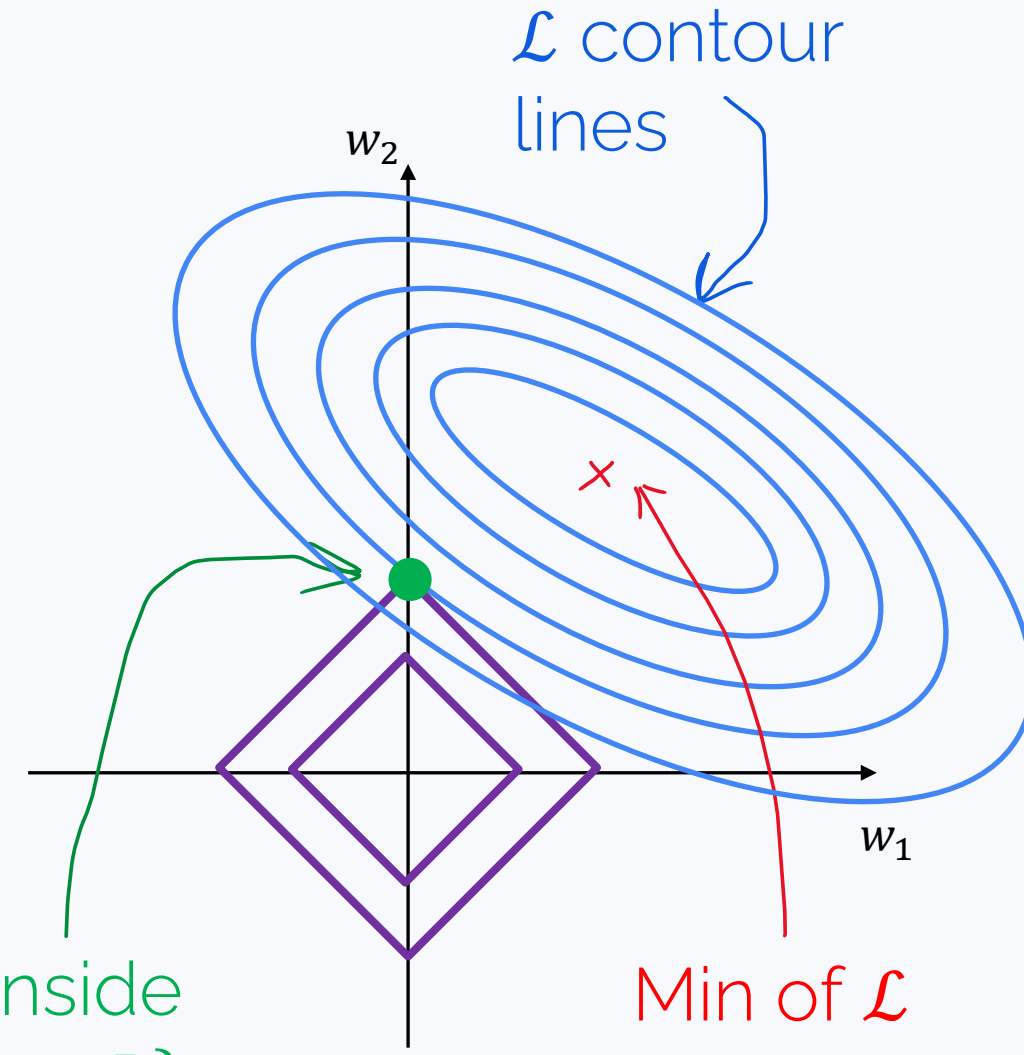


L1-regularization

$$\mathcal{L}_{reg}(y, X, w) = \mathcal{L}(y, X, w) + \lambda \|w\|_1$$

where $\|w\|_1 = |w_1| + \cdots + |w_D|$

Theory says that optimizing this new loss is equivalent to optimizing $\mathcal{L}(y, X, w)$ inside some $\{\|w\|_1 \leq R\}$



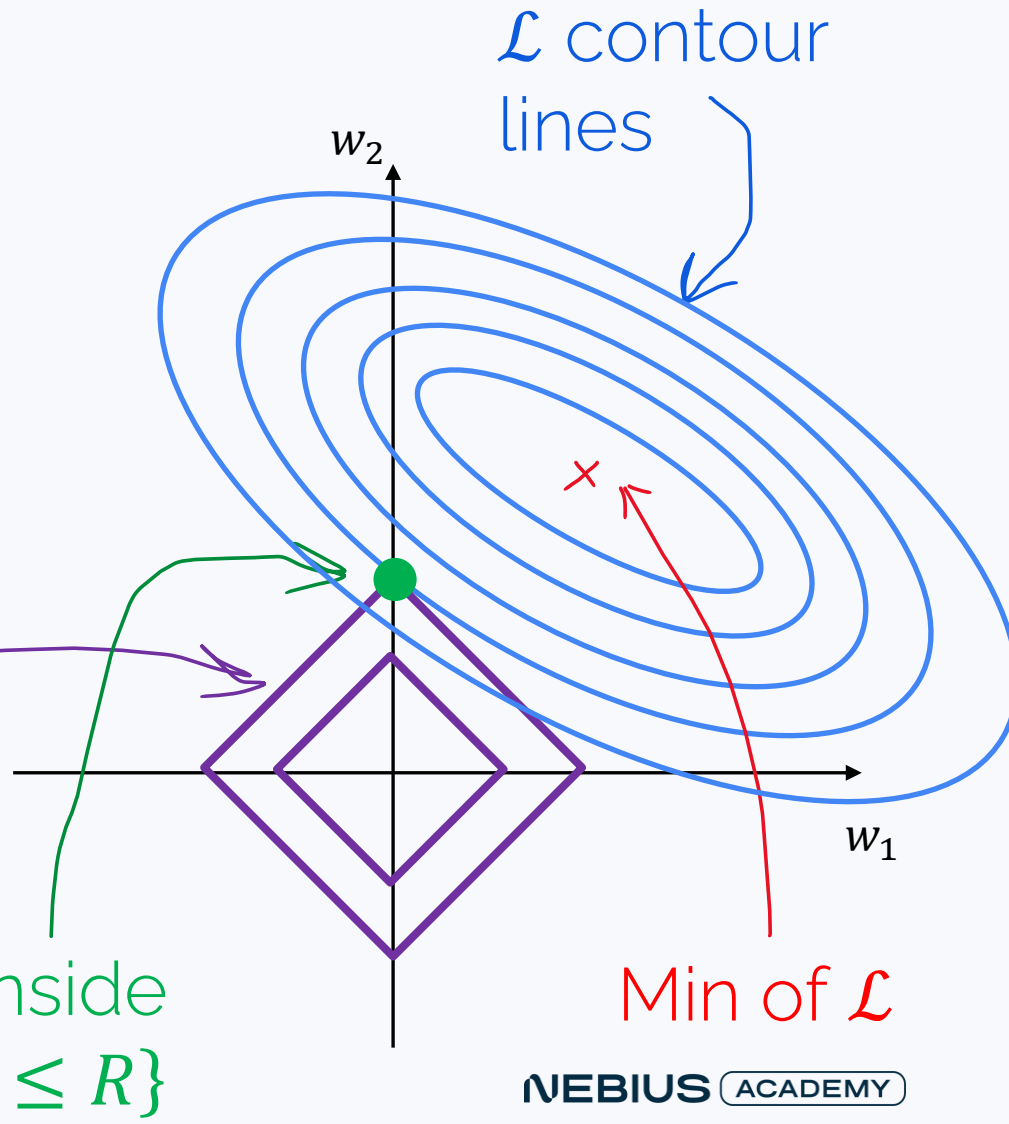
L1-regularization leads to sparsity

$$\mathcal{L}_{reg}(y, X, w) = \mathcal{L}(y, X, w) + \lambda \|w\|_1$$

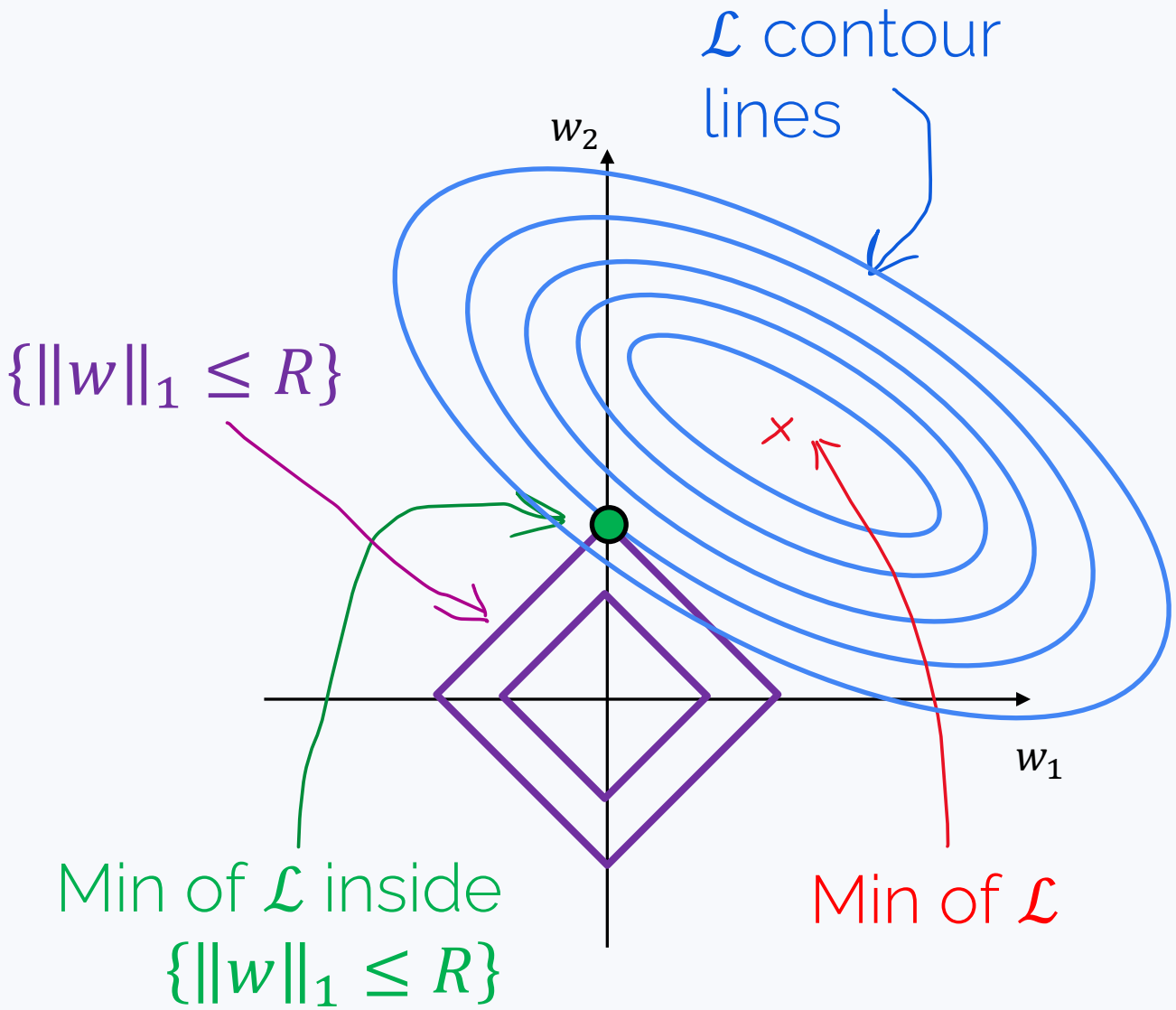
where $\|w\|_1 = |w_1| + \cdots + |w_D|$

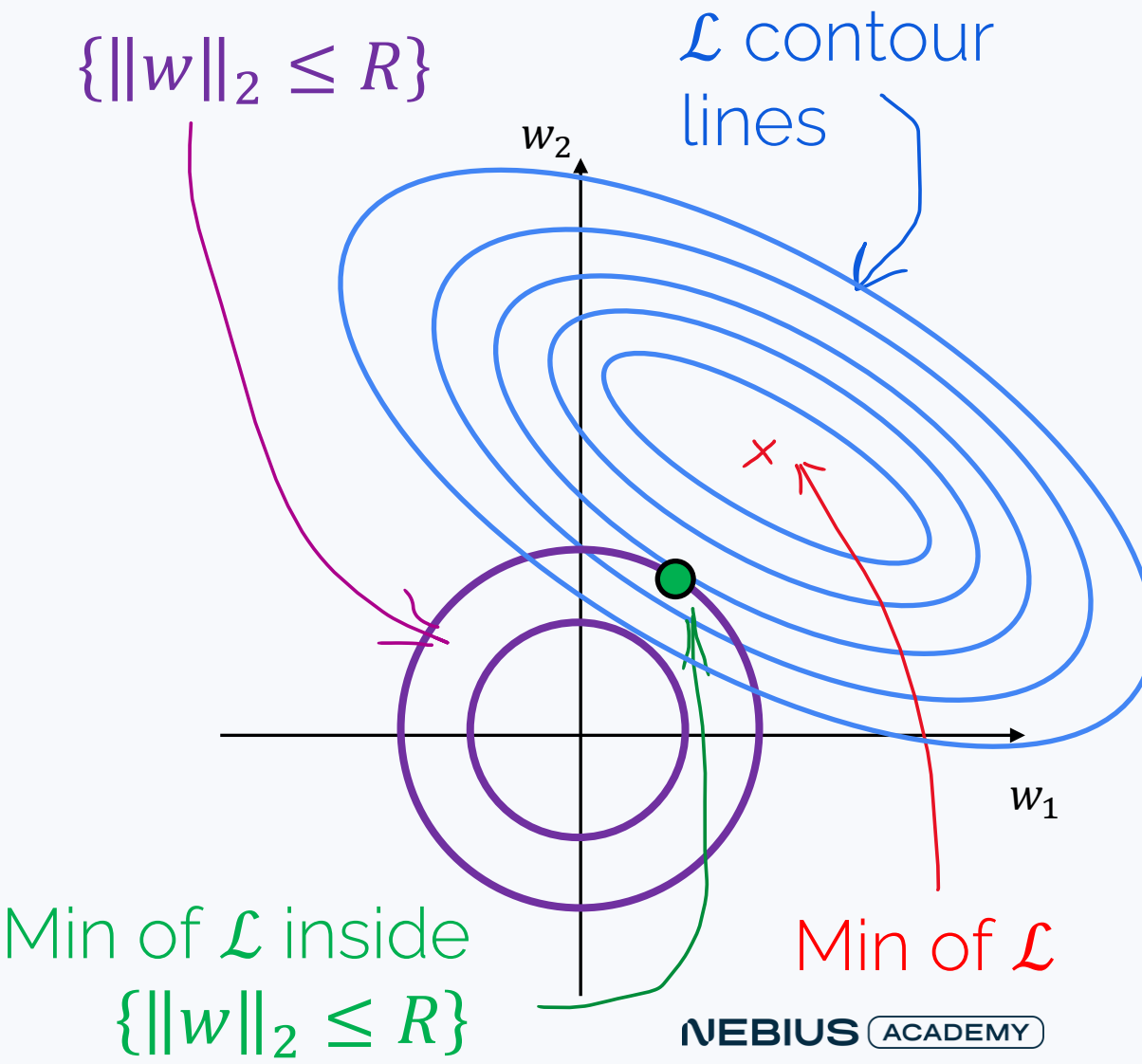
Theory says that optimizing this new loss is equivalent to optimizing $\mathcal{L}(y, X, w)$ inside some $\{\|w\|_1 \leq R\}$

Many coordinates of the resulting $\widehat{w}$ will likely be zero.
The larger λ, the more.



Compare: L1 vs L2





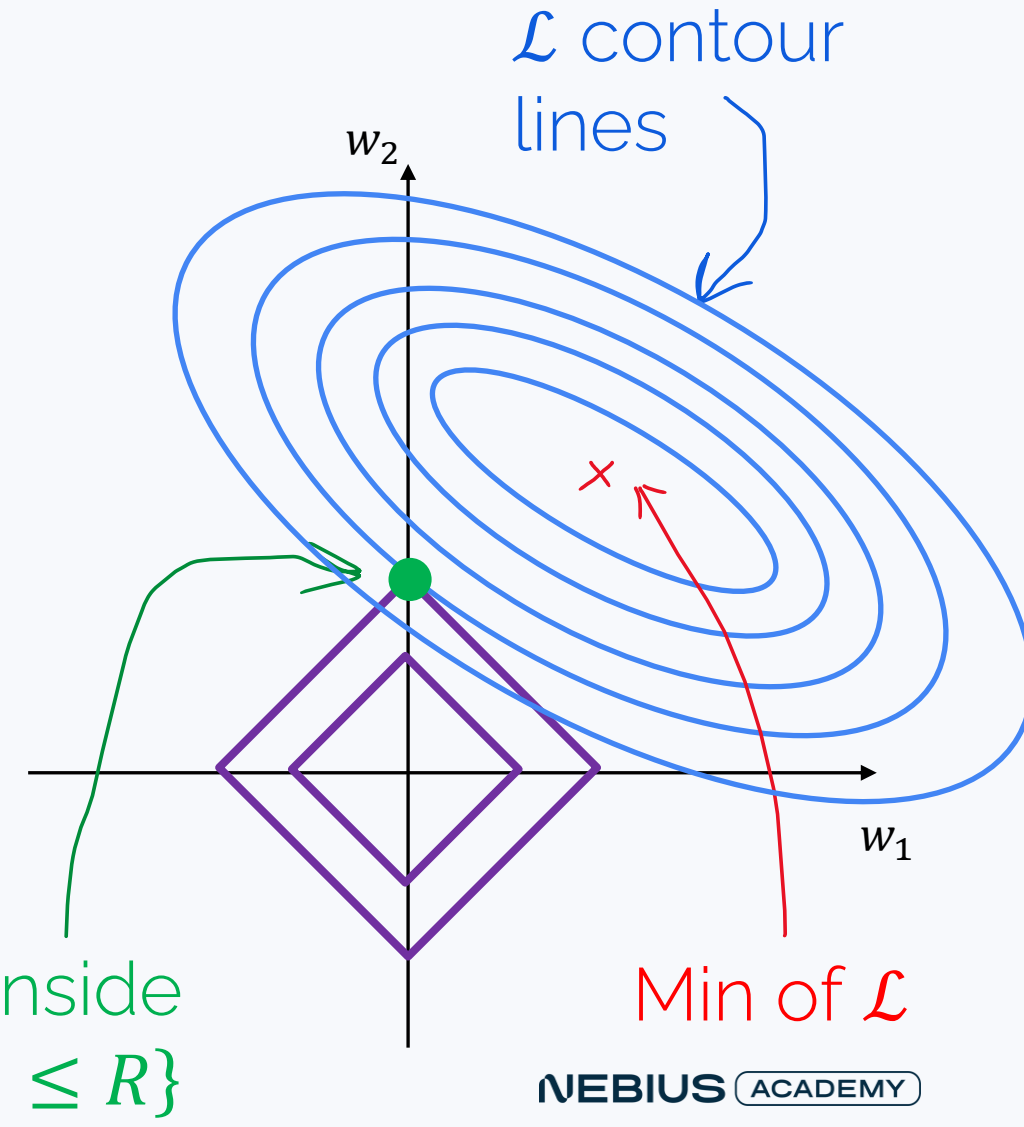
L1-regularization: why sparsity?

$$\mathcal{L}_{reg}(y, X, w) = \mathcal{L}(y, X, w) + \lambda \|w\|_1$$

where $\|w\|_1 = |w_1| + \cdots + |w_D|$

Many coordinates of the resulting $\hat{w}$ will likely be zero.

Why bother? There's hope that we'll get rid of noise and irrelevant features. Counters overfitting.



Adam with L_2-regularization

$$g_k = \nabla_w \mathcal{L}(w_k) + \lambda w_k$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

Accumulates previous directions
Makes GD less erratic

$$\nu_k = \beta_2 \cdot \nu_{k-1} + (1 - \beta_2) g_k^2$$

Adaptive step
for each coordinate
(~second-order methods)

$$\hat{\mu}_k = \frac{\mu_k}{1 - \beta_1^k}, \qquad \hat{\nu}_k = \frac{\nu_k}{1 - \beta_2^k}$$

Bias correction

$$w_k = w_{k-1} - \alpha_k \cdot \frac{\hat{\mu}_k}{\sqrt{\hat{\nu}_k + \epsilon}}$$

AdamW: Adam with L_2 weight decay

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

Accumulates previous directions
Makes GD less erratic

$$\nu_k = \beta_2 \cdot \nu_{k-1} + (1 - \beta_2) g_k^2$$

Adaptive step
for each coordinate
(~second-order methods)

$$\hat{\mu}_k = \frac{\mu_k}{1 - \beta_1^k}, \qquad \hat{\nu}_k = \frac{\nu_k}{1 - \beta_2^k}$$

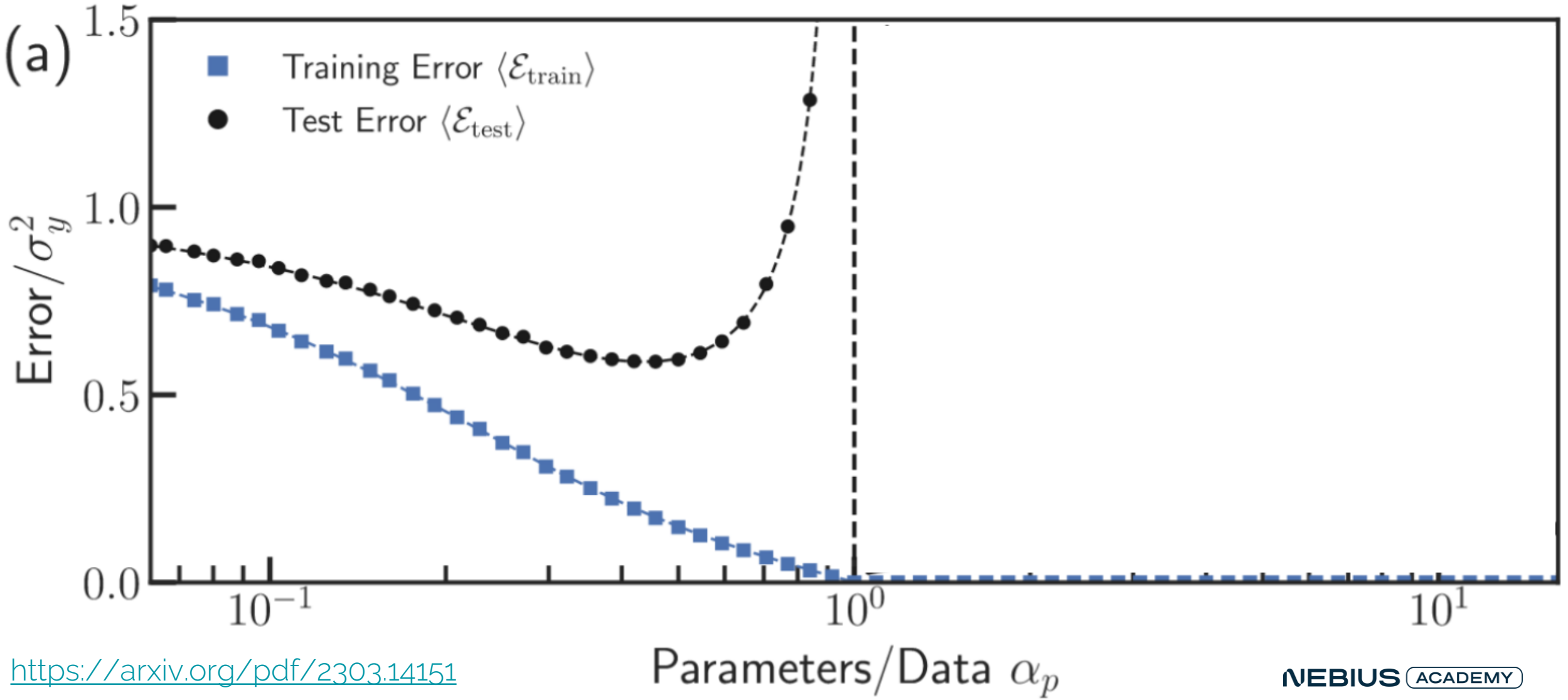
Bias correction

$$w_k = w_{k-1} - \alpha_k \cdot \left(\frac{\hat{\mu}_k}{\sqrt{\hat{\nu}_k + \epsilon}} + \lambda w_k \right)$$

NEBIUS ACADEMY

Complexity and overfitting

A classical view on complexity vs generalization



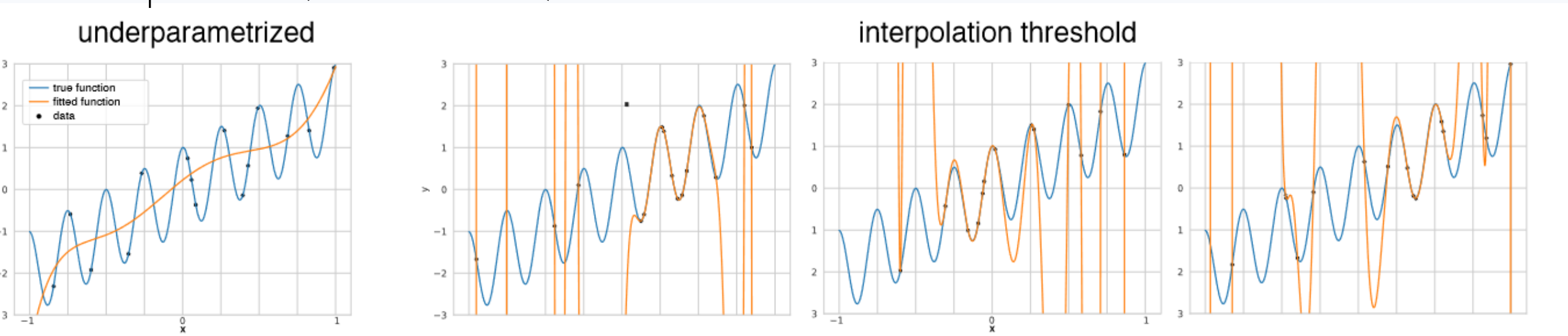
https://arxiv.org/pdf/2303.14151

Approximation vs interpolation

N points, D functions

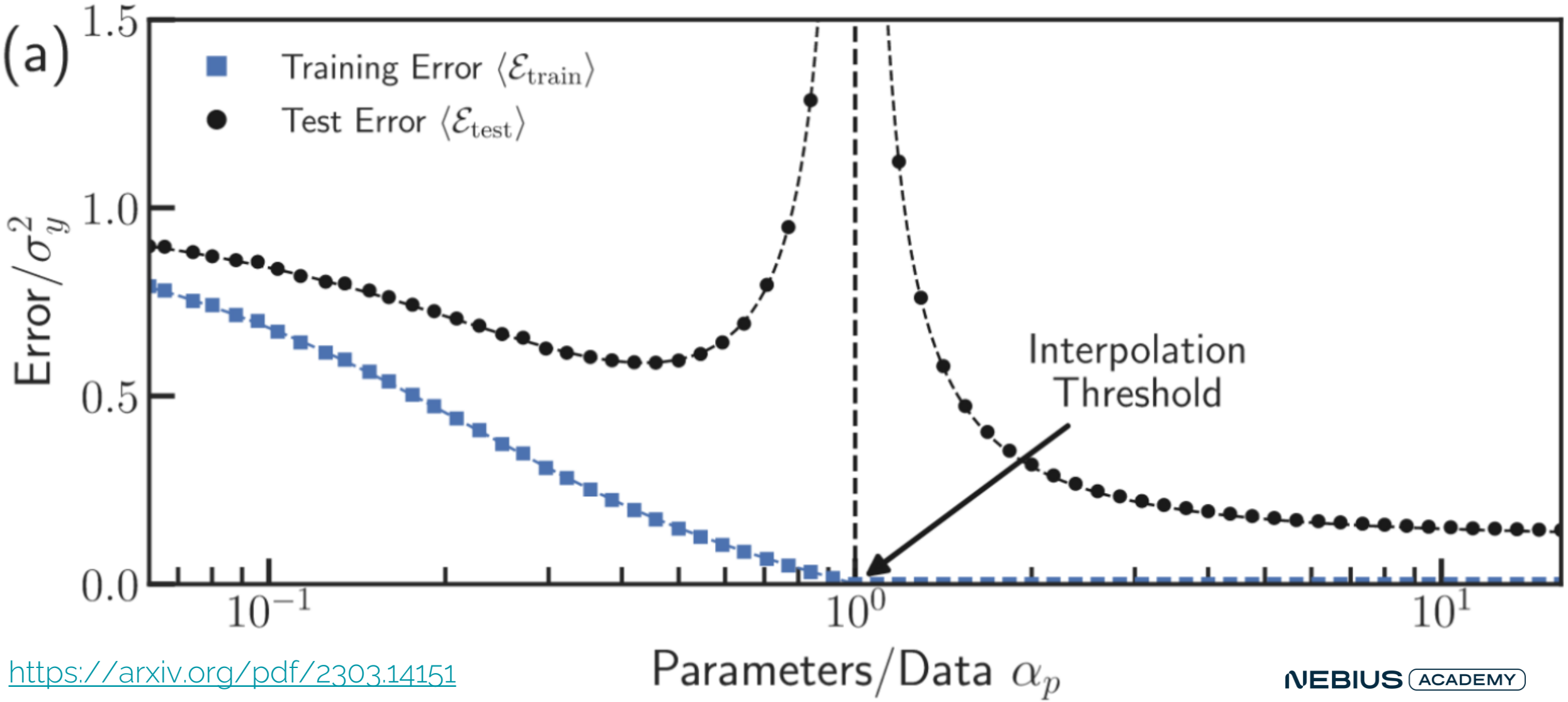
$$\hat{w} = \left(X^T X\right)^{-1} X^T y$$

Interpolation (train error = 0) for $N = D$



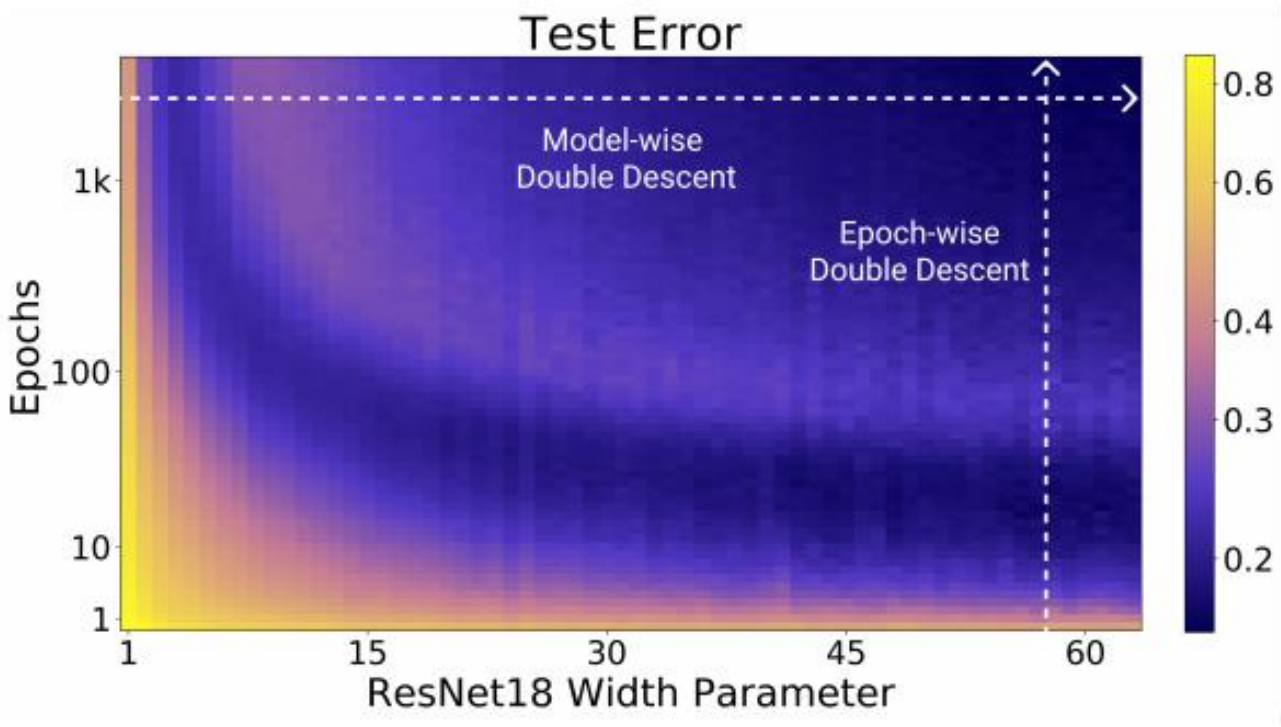
https://arxiv.org/pdf/2303.14151

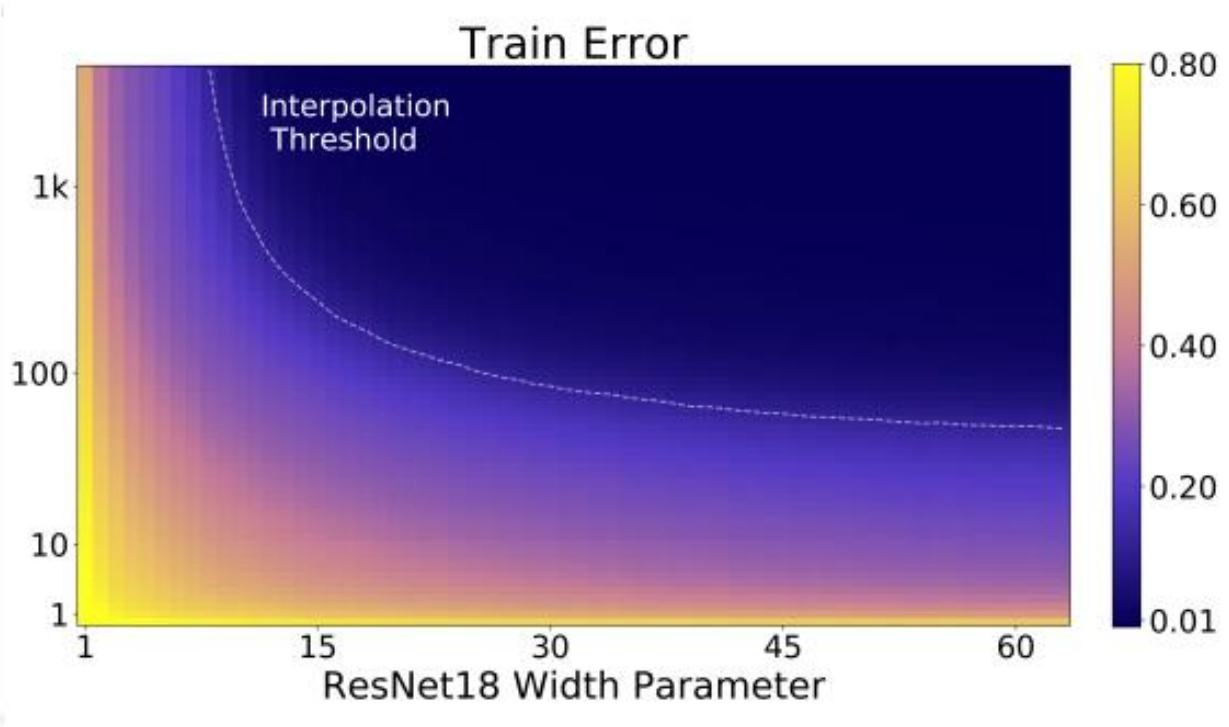
Double descent



https://arxiv.org/pdf/2303.14151

Double descent





https://www.lesswrong.com/posts/FRv7ryoqtvSuqBxuT/understanding-deep-double-descent
https://openai.com/index/deep-double-descent/

Approximation vs interpolation

N points, D functions

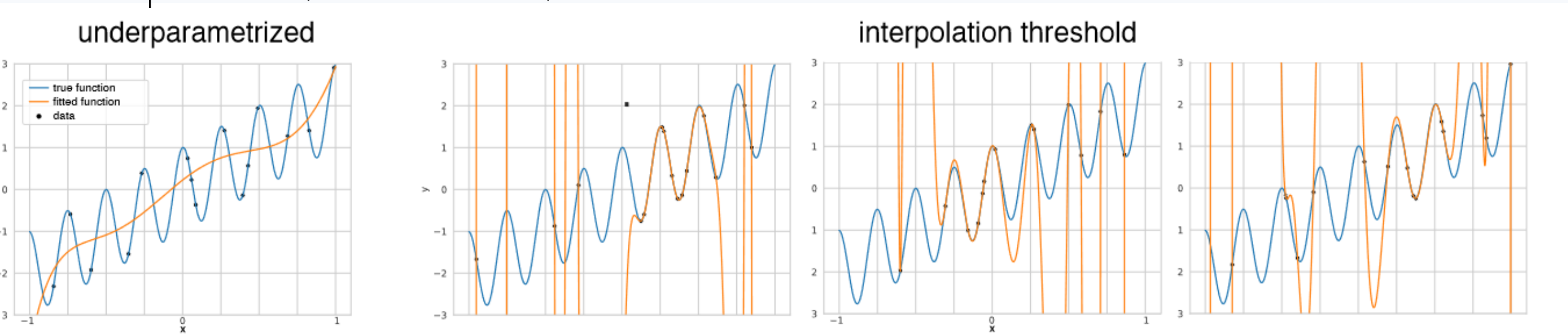
$$\hat{w} = \left(X^T X\right)^{-1} X^T y$$

When $D > N$, solve

$$\begin{cases} \|w\| \to \min \\ Xw = y \end{cases}$$

$$\hat{w} = X^T \left(XX^T\right)^{-1} y$$

Interpolation (train error = 0) for $N = D$



https://arxiv.org/pdf/2303.14151

Sharp and flat minima

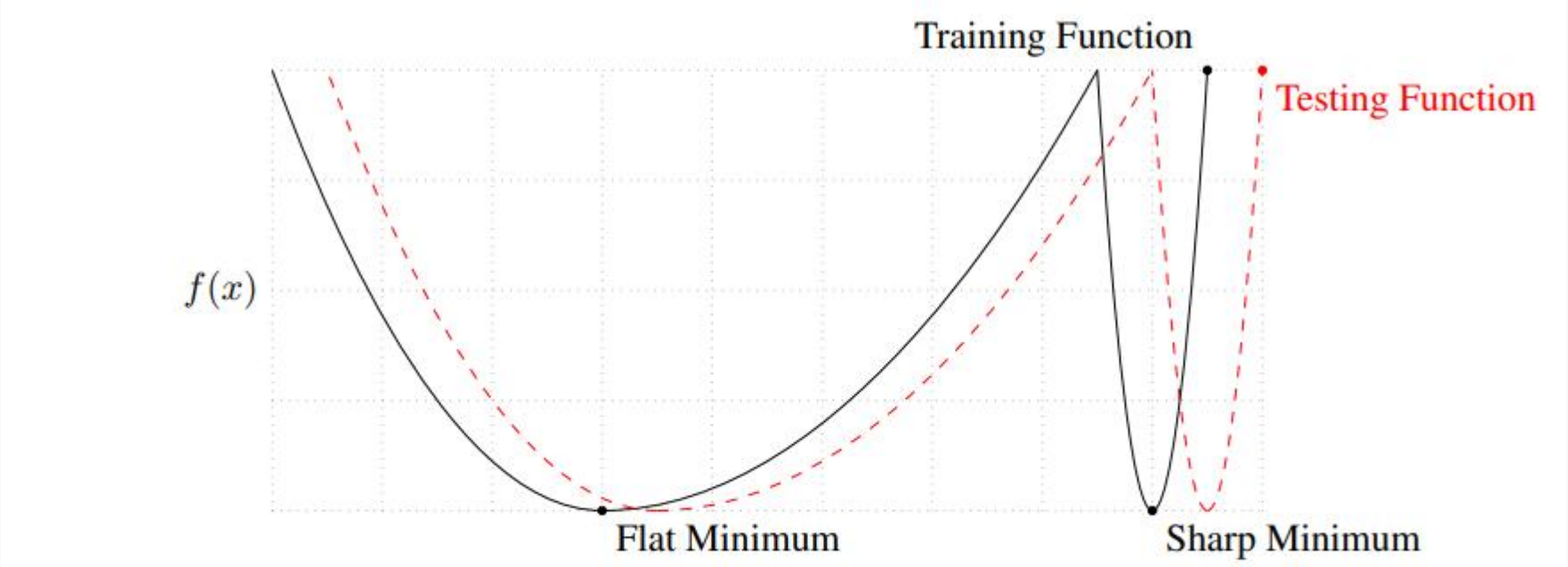


Figure 1: A Conceptual Sketch of Flat and Sharp Minima. The Y-axis indicates value of the loss function and the X-axis the variables (parameters)

https://arxiv.org/pdf/1609.04836

On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima

Overparametrized optimization

$$\left\| y - Xw^T \right\|_2^2 \to \min_w$$

Overparametrized optimization: we have so many features that we can just memorize the training data.

Example. 1d polynomial regression, $D > N$. Then

- Features are linearly dependent,
- $X^T X$ is non-invertible,
- We can just fit the curve through the training points,
- And not just one but many curves.

Overparametrized optimization

$$\left\|y - Xw^T\right\|_2^2 \to \min_w$$

Overparametrized optimization: we have so many features that we can just memorize the training data.

Solve instead:

$$\begin{cases} \|w\|_2^2 \to \min \\ Xw^T = y \end{cases}$$

Solution: $w^T = X^T\left(XX^T\right)^{-1}y$. ($XX^T$ is invertible.)